FLOW NETWORKS AND MAX FLOW

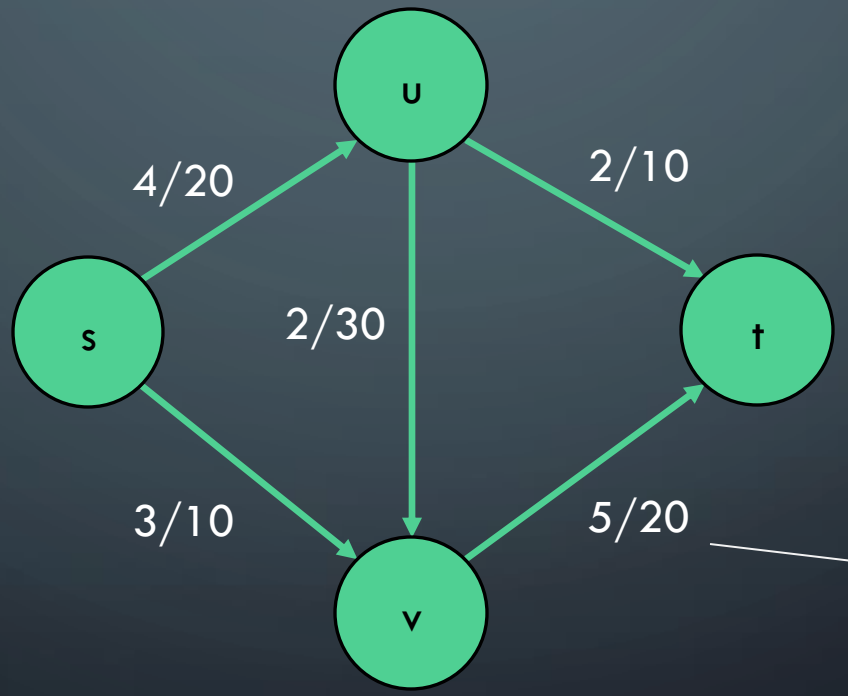
DATA STRUCTURES AND ALGORITHMS 2

MARK FLORYAN

FLOW NETWORKS

FLOW NETWORKS

A ***flow network*** is a specialized directed graph with a single source and single sink (terminus). Each edge has a capacity of flow that can be carried on that edge.



Source node produces flow

Sink (terminus) node consumes flow

Each edge has a capacity (second number) and current flow (first number)

What is the maximum flow you can send from s to t?

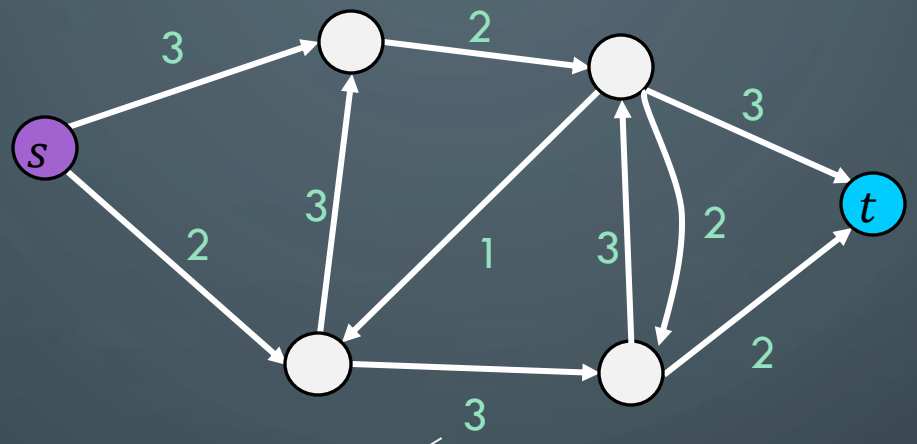
APPLICATIONS

- Transportation networks
  - How many people can be routed?
- Computer networks
- Electrical distribution
- Water distribution

- Note that all these applications have multiple sources and multiple sinks!
  - Whereas the flow networks we study do not, yet

ANOTHER EXAMPLE OF FLOW NETWORK

Graph $G = (V, E)$



Source node $s \in V$

Sink node $t \in V$

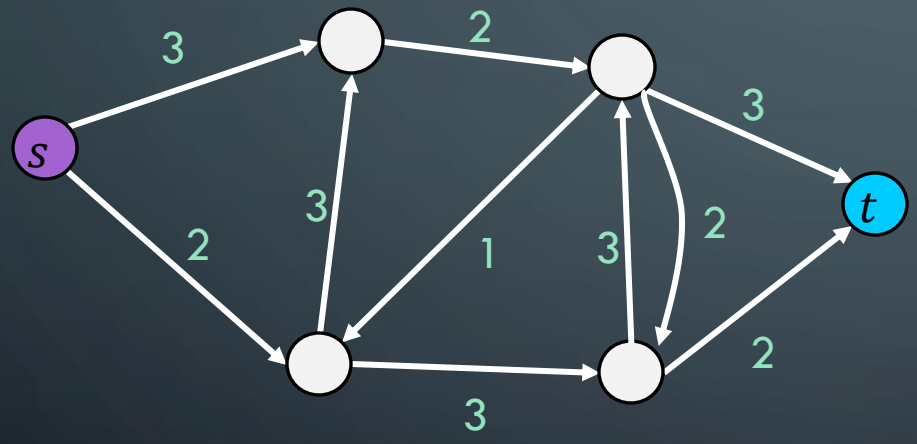
Edge Capacities $c(e) \in \mathcal{N}$

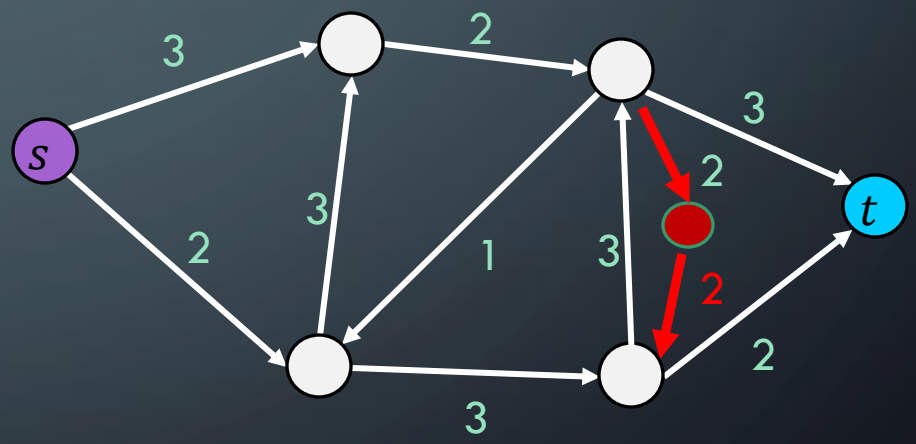
If $(u, v) \in E$ then $(v, u) \notin E$ (Note our example here violates this!)

Max flow intuition: If s is a faucet, t is a drain, and s connects to t through a network of pipes with given capacities, what is the maximum amount of water which can flow from the faucet to the drain?

FLOW NETWORK: REMOVING ANTIPARALLEL EDGES

Easy adjustment to remove antiparallel edges and have equivalent flow graph: add intermediate node





FLOW

Flow is an assignment of integer flow values to edges that follows all of the constraints of flow networks. For each edge, $f(e) = n$

Flow constraint

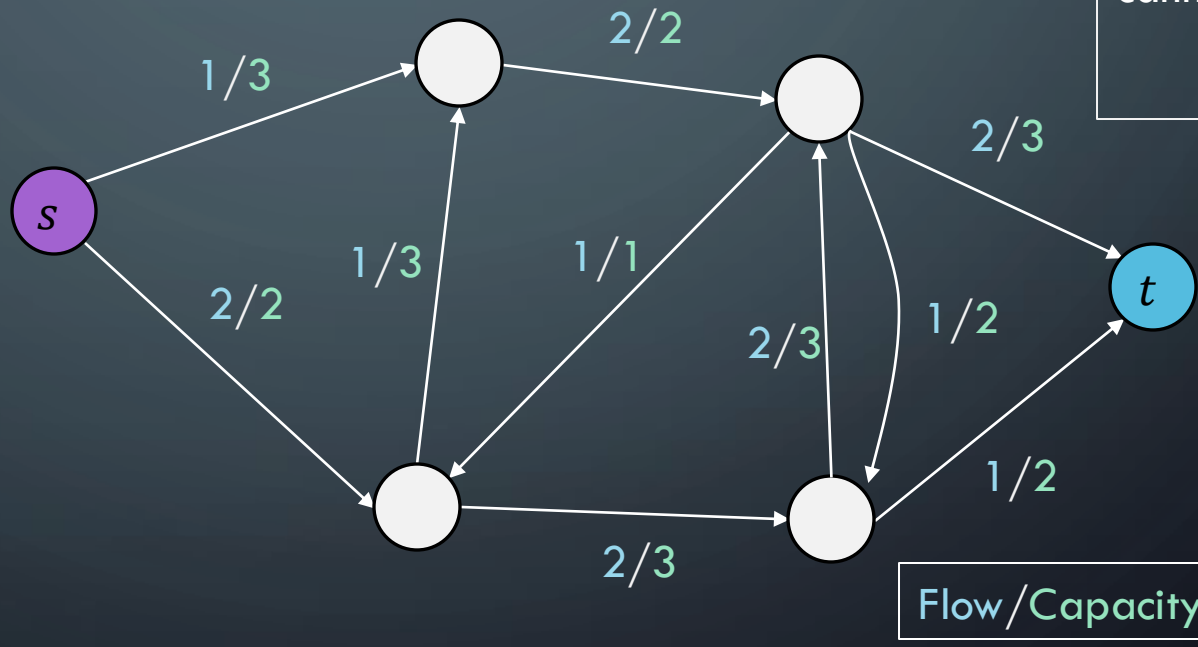
$\forall v \in V - \{s, t\}, inflow(v) = outflow(v)$

$$inflow(v) = \sum_{x \in V} f(x, v)$$

$$outflow(v) = \sum_{x \in V} f(v, x)$$

Capacity constraint – Flow cannot exceed capacity:

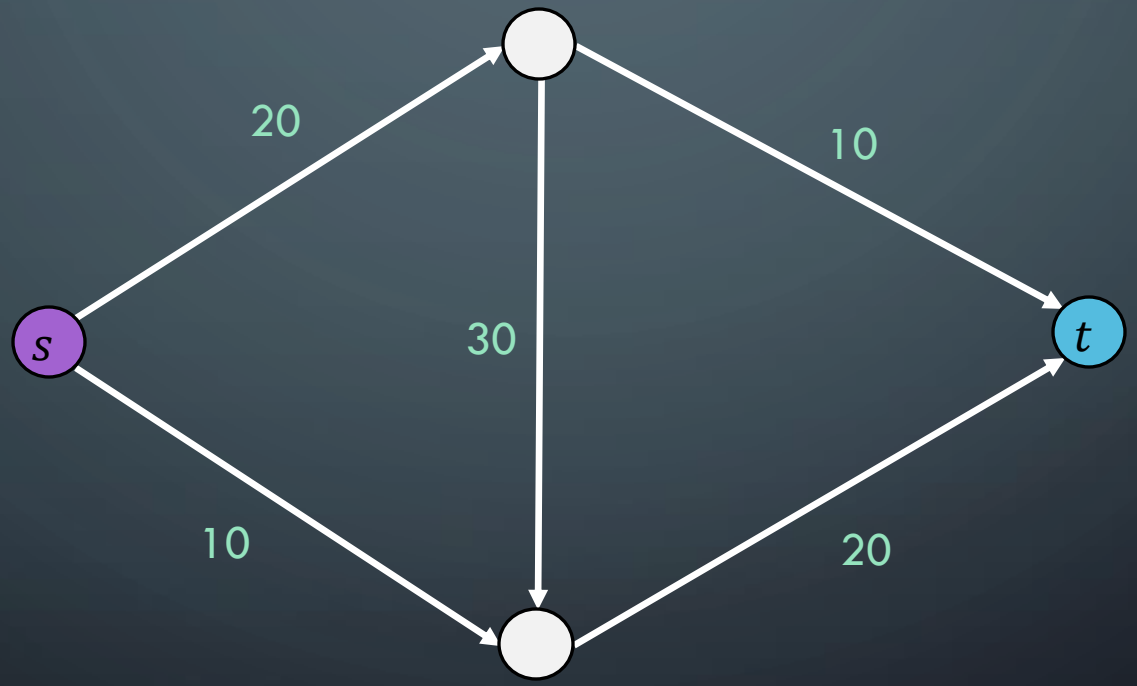
$f(e) \leq c(e)$



Total flow of G is net flow out of s: $|f| = outflow(s) - inflow(s)$

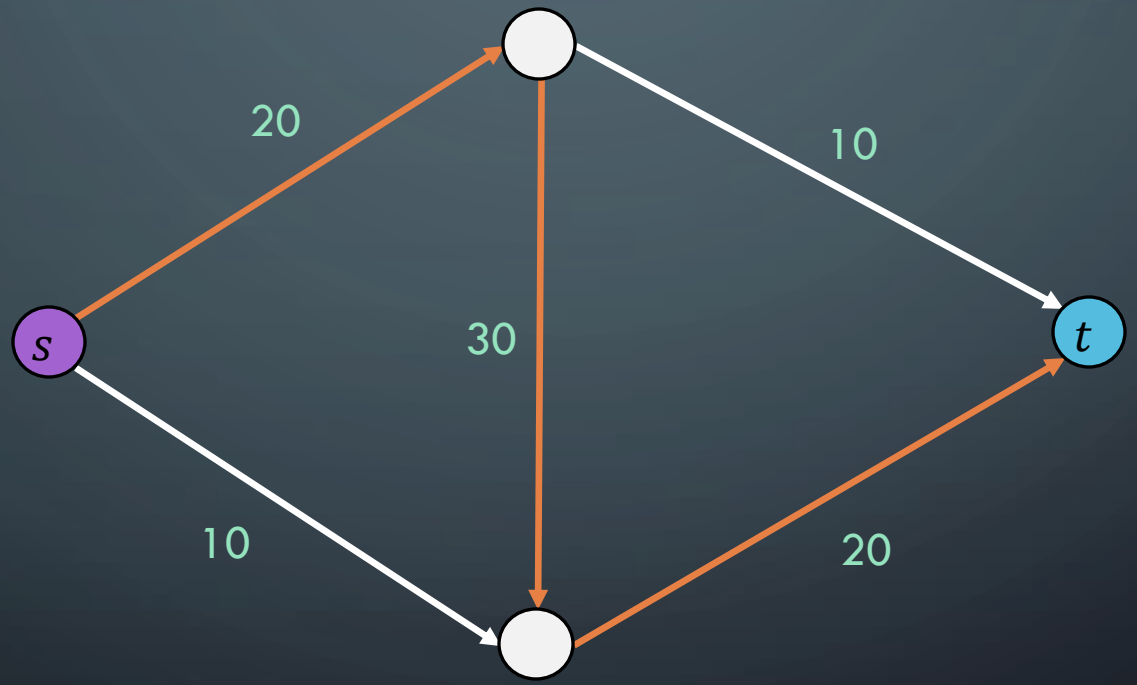
CAN GREEDY ALGORITHM FIND MAX FLOW

Possible Greedy Choice: Saturate Highest Capacity Path First



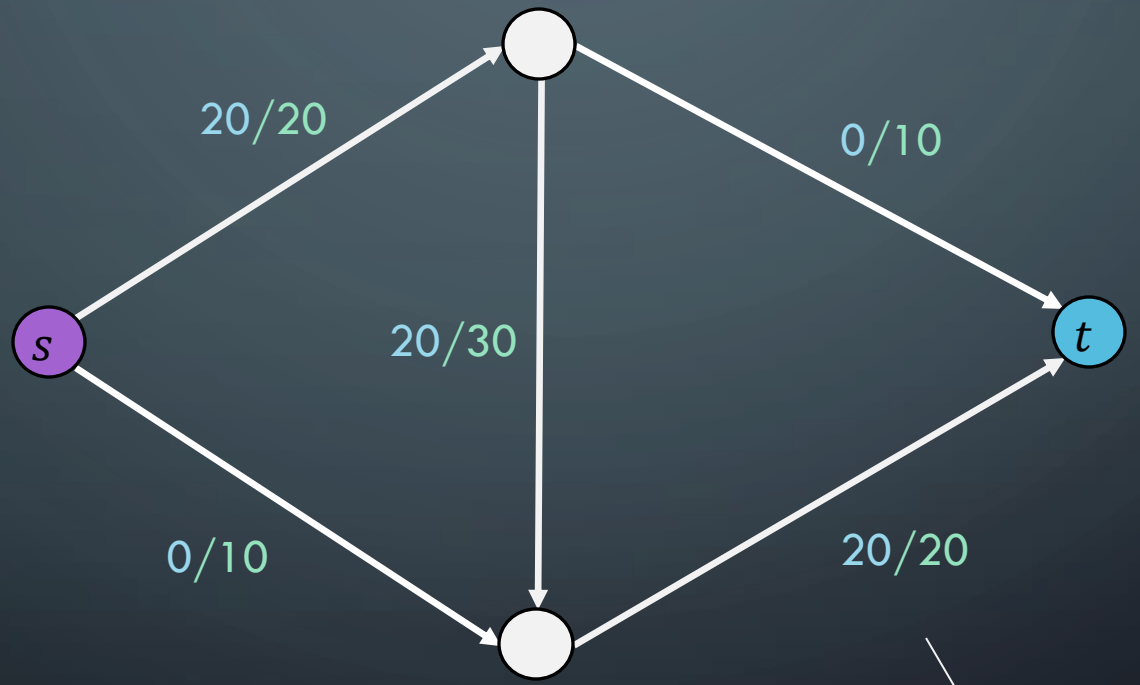
CAN GREEDY ALGORITHM FIND MAX FLOW

Possible Greedy Choice: Saturate Highest Capacity Path First



CAN GREEDY ALGORITHM FIND MAX FLOW

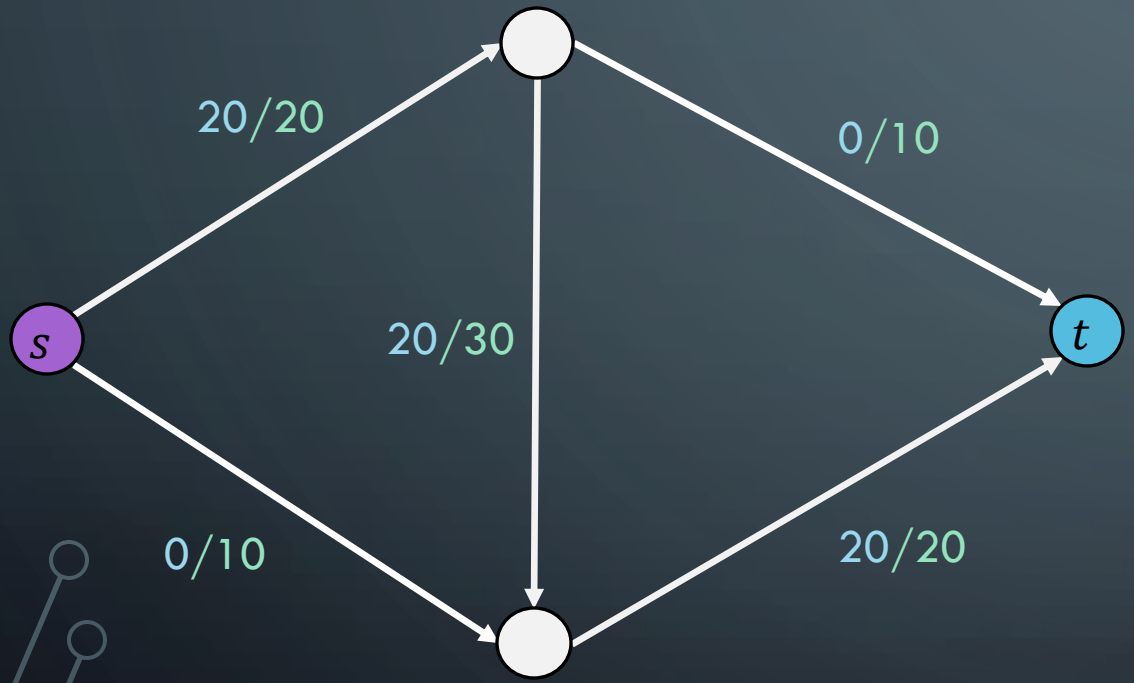
Possible Greedy Choice: Saturate Highest Capacity Path First



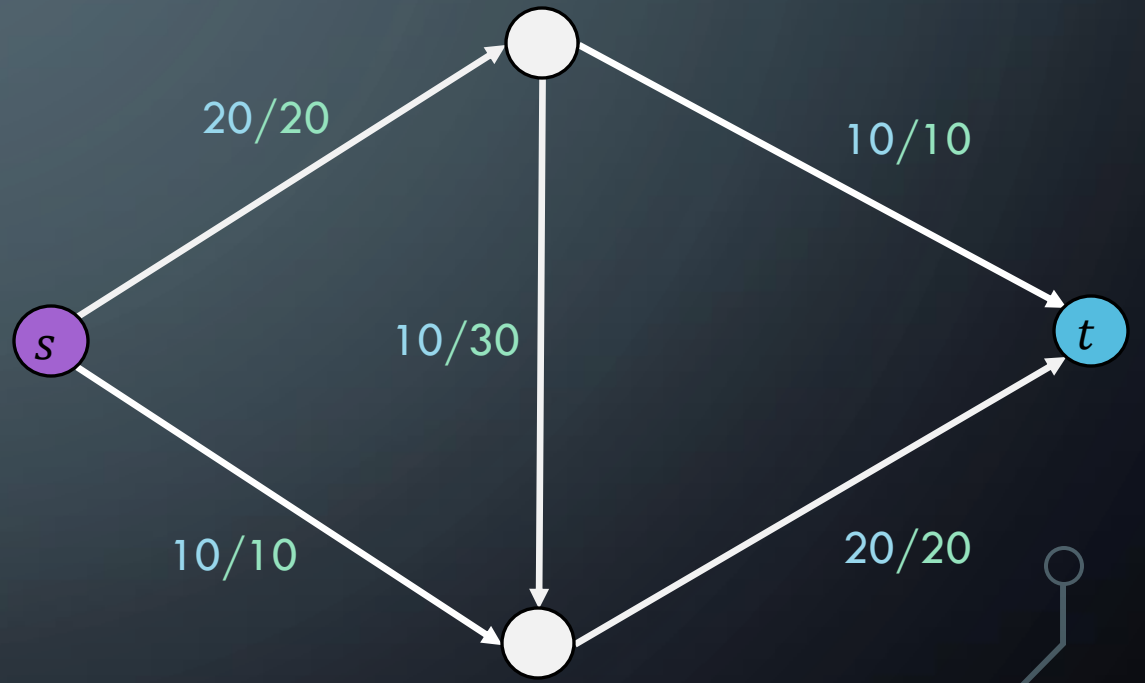
Greedy gets a solution of $|f| = 20$

CAN GREEDY ALGORITHM FIND MAX FLOW

Possible Greedy Choice: Saturate Highest Capacity Path First



Greedy gets a solution of $|f| = 20$



But $|f| = 30$ is better

SOLVING FLOW NETWORKS
FORD-FULKERSON

FORD-FULKERSON: ALGORITHM OVERVIEW

Overall Idea:

```
Do until not possible anymore:
    Find a path through the graph we can push flow through
        This path might need to "undo" past choices
    Push as much flow through that path as possible
    Repeat
```

Concerns:
How to represent the graph?
How to find safe paths to push flow through?
How to undo past choices that were suboptimal?

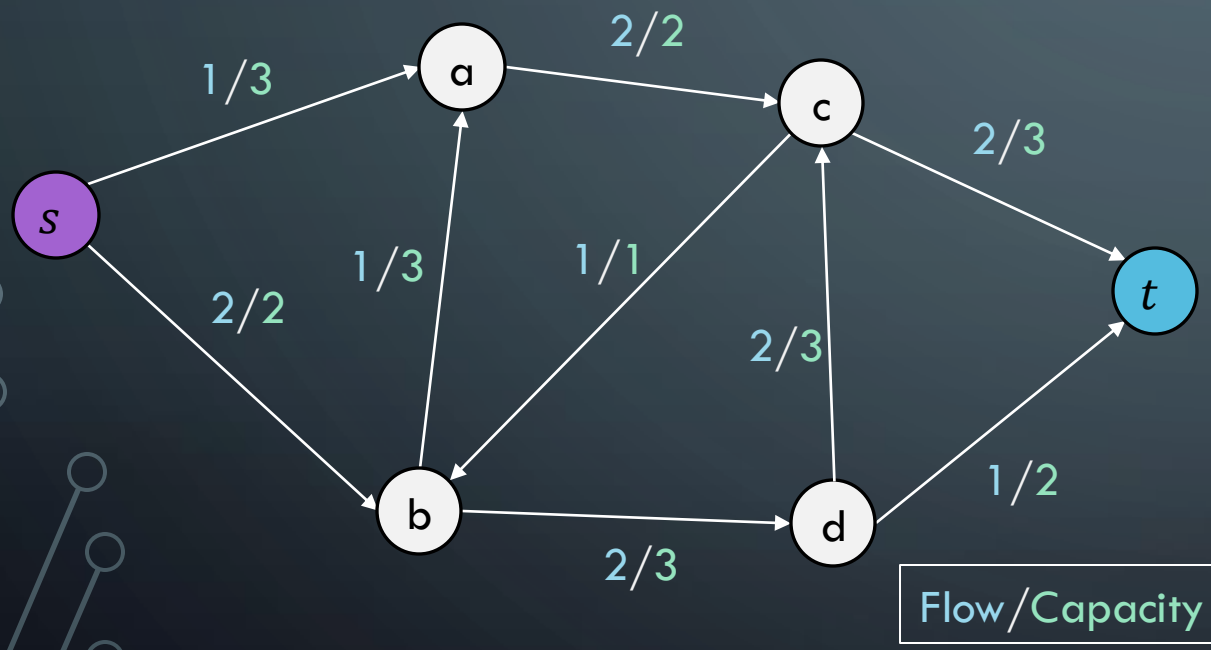
ALGORITHM NOTATION

- f(u,v): the flow on the edge from u to v
- f(v,u): the backflow on the edge from v to u
- c(u,v): the capacity on the edge from u to v
- $c_f(u,v)$: the *residual* capacity on the edge from u to v
- G_f is the graph where the edges weights are the residual capacities
  - THIS is usually the graph we actually use when running the algorithm we are about to see.

RESIDUAL GRAPH

We are going to represent the flow network (graph) using a "residual graph". This graph is an adjacency matrix storing the residual capacities at each edge.

Original Graph



Residual Graph

	s	a	b	c	d	t
s	—	2	0	—	—	—
a	—	—	—	0	—	—
b	—	2	—	—	1	—
c	—	—	0	—	—	1
d	—	—	—	1	—	1
t	—	—	—	—	—	—

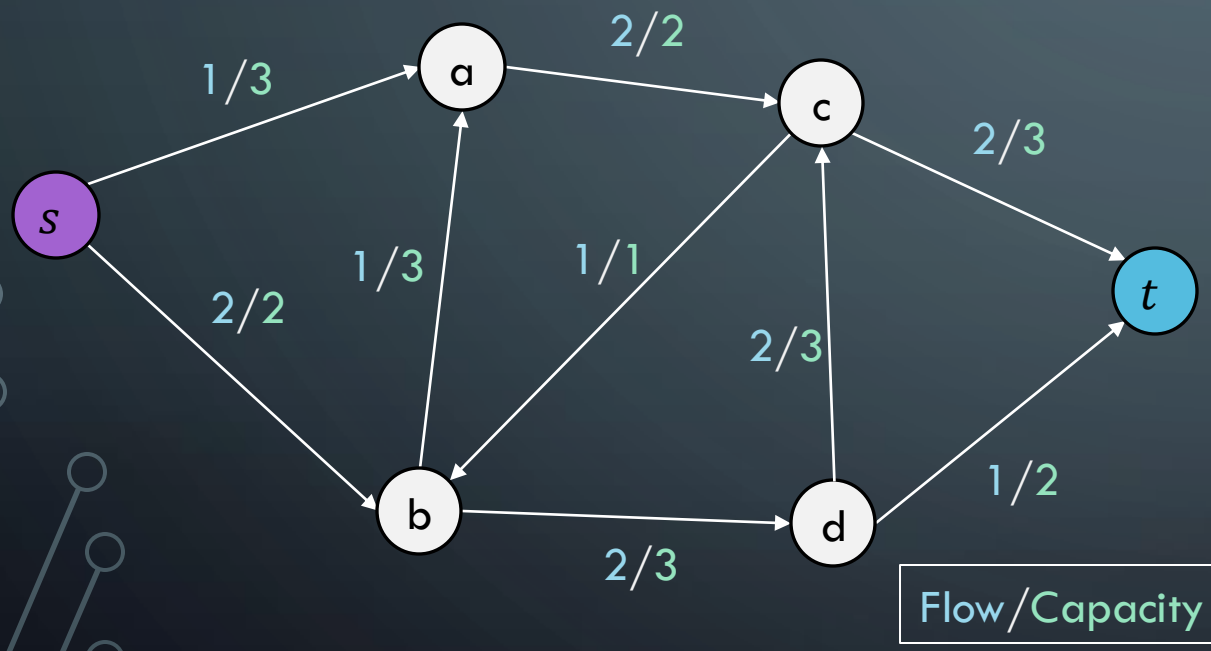
Residual Capacity is room left until full: $c_f(u, v) = c(u, v) - f(u, v)$

BACKFLOW

The weights highlighted in blue in the residual graph are called backflow edges.

These represent flow that can be "removed" on an edge

Original Graph



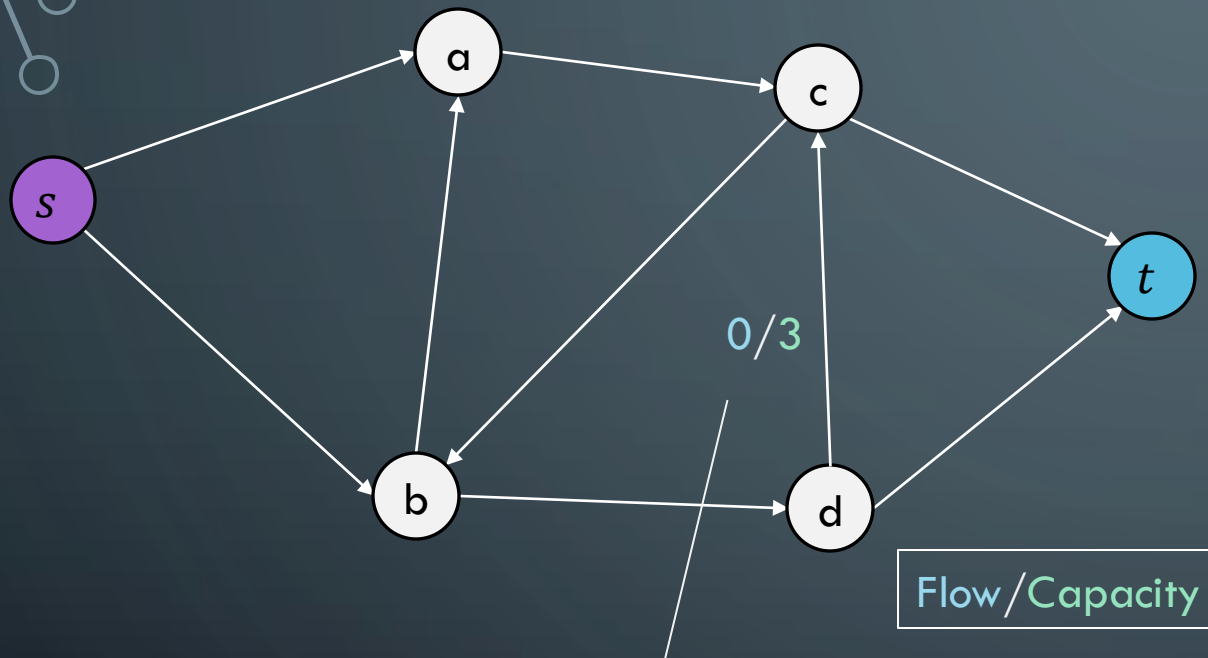
Residual Graph

	s	a	b	c	d	t
s	—	2	0	—	—	—
a	1	—	1	0	—	—
b	2	2	—	1	1	—
c	—	2	0	—	2	1
d	—	—	2	1	—	1
t	—	—	—	2	1	—

Backflow edges are treated the same, they represent flow that can be pushed through that edge!

BACKFLOW EXAMPLE

Original Graph



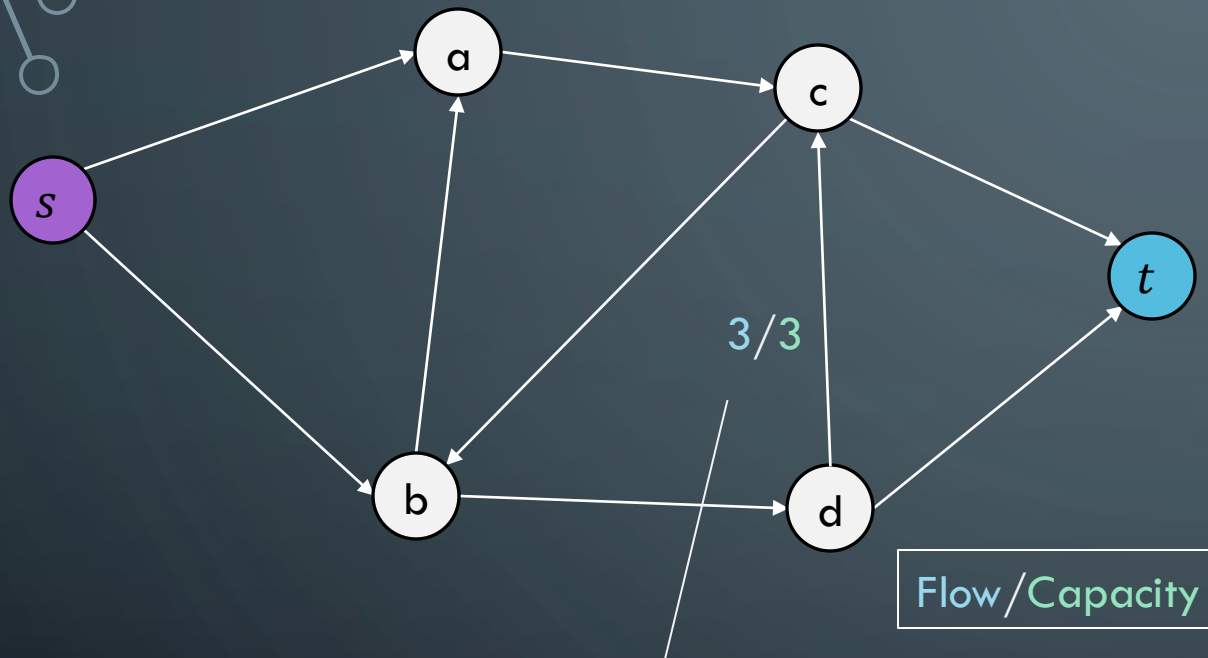
Residual Graph

	s	a	b	c	d	t
s	—	—	—	—	—	—
a	—	—	—	—	—	—
b	—	—	—	—	—	—
c	—	—	—	—	0	—
d	—	—	—	3	—	—
t	—	—	—	—	—	—

When no flow is being pushed on this edge the residual capacity is 3 because we can push up to 3 more units on this edge. The backflow capacity is 0 because we can't undo any of the flow.

BACKFLOW EXAMPLE

Original Graph



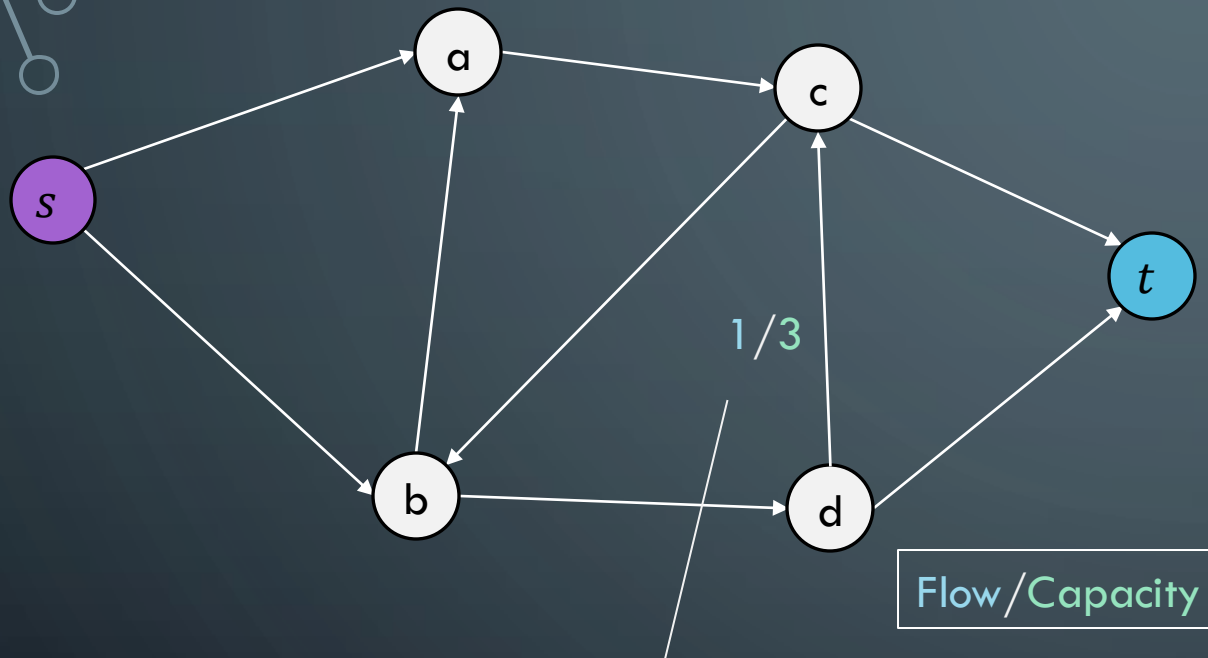
Residual Graph

	s	a	b	c	d	t
s	—	—	—	—	—	—
a	—	—	—	—	—	—
b	—	—	—	—	—	—
c	—	—	—	—	3	—
d	—	—	—	0	—	—
t	—	—	—	—	—	—

Let's say we end up pushing 3 units of flow on this edge. Now the residual capacity is 0 (no more room) but the backflow capacity is 3 (can undo up to 3 units)

BACKFLOW EXAMPLE

Original Graph



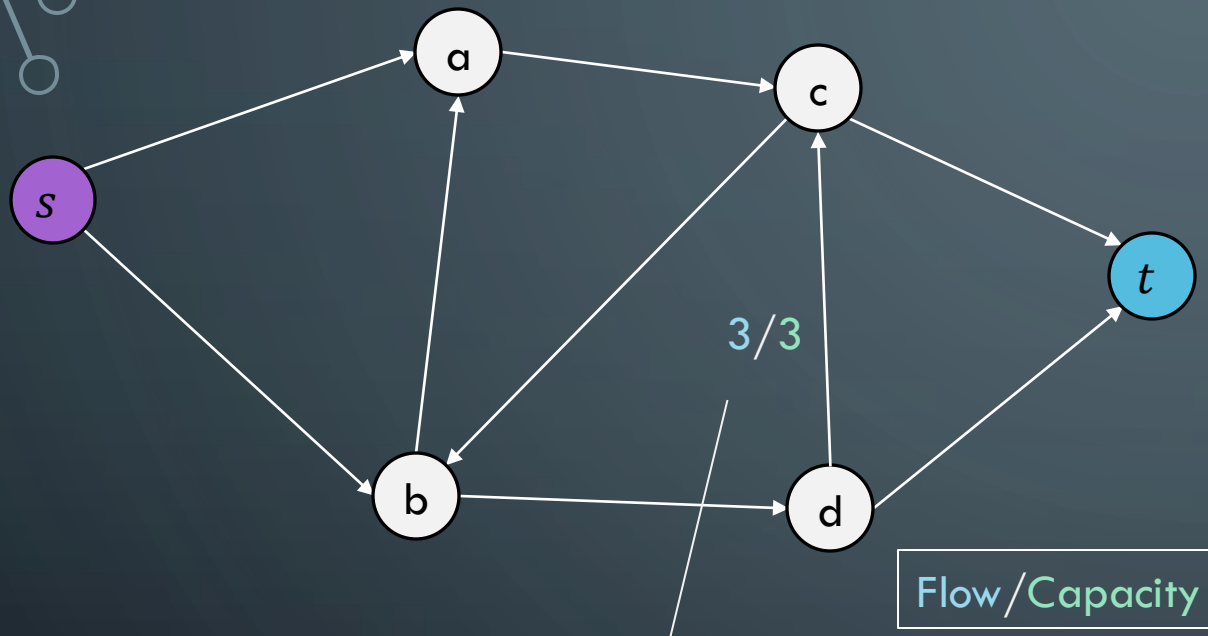
Residual Graph

	s	a	b	c	d	t
s	—	—	—	—	—	—
a	—	—	—	—	—	—
b	—	—	—	—	—	—
c	—	—	—	—	1	—
d	—	—	—	2	—	—
t	—	—	—	—	—	—

Now let's say we "push" 2 units of flow down the backflow edge. This effectively removes two units of flow from the main edge. Our residual capacity becomes 2, our backflow capacity becomes 1

BACKFLOW EXAMPLE

Original Graph



Flow/Capacity

Residual Graph

	s	a	b	c	d	t
s	—	—	—	—	—	—
a	—	—	—	—	—	—
b	—	—	—	—	—	—
c	—	—	—	—	3	—
d	—	—	—	0	—	—
t	—	—	—	—	—	—

The algorithm will treat forward and back edges the same. No idea to distinguish between them! Neat!

Removing capacity from an edge or back edge always adds that capacity to the opposite paired edge

An edge and its back edge residual capacities will always add up to the capacity of the edge in the graph!

FORD-FULKERSON ALGORITHM

Augmenting Path: A path through the graph from source node to sink node such that each edge along the path has a positive, non-zero residual capacity

Note that augmenting paths might include backflow edges, meaning flow is being removed from some edge!

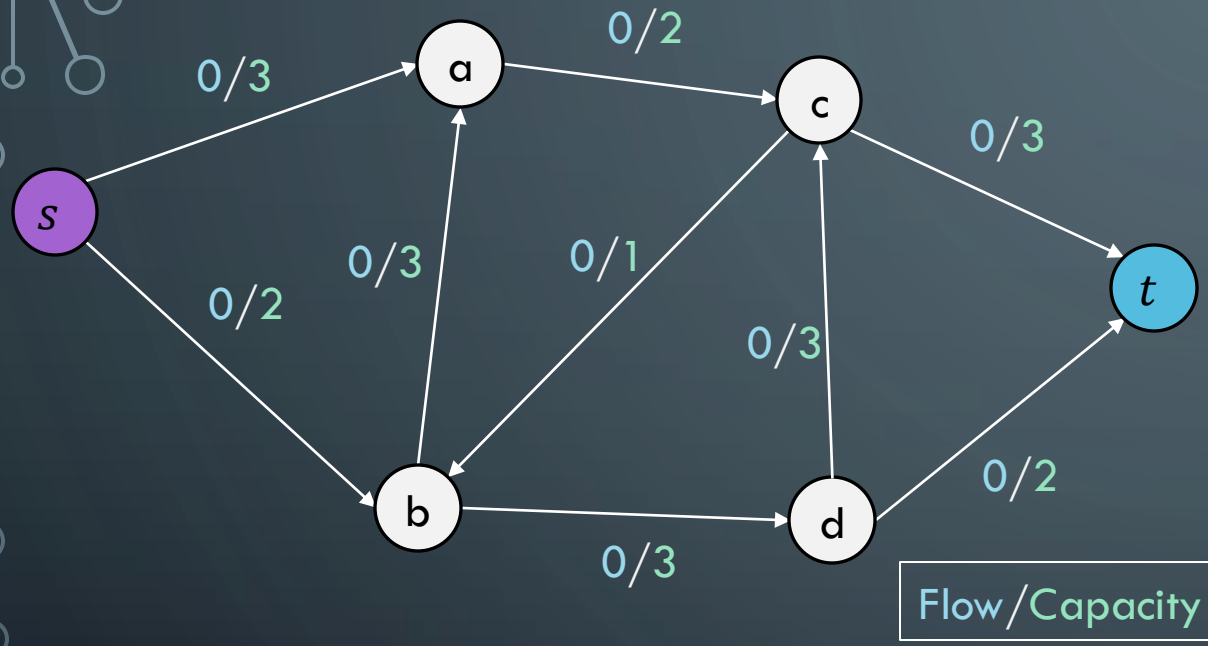
Algorithm idea: Keep finding augmenting paths until we can't push anymore flow through the network.

FORD-FULKERSON ALGORITHM:

```
Ford-Fulkerson(G, s, t):
    Construct residual graph G_f with f(u,v) = 0 for all edges
    maxflow = 0
    while augmenting paths exist:
        Run DFS to find path p in G_f | c_f(u,v) > 0 for all edges (u,v) ∈ p
        Find c_f(p) = min{c_f(u,v) | (u,v) ∈ p}
        for each edge (u,v) ∈ p:
            G_f[u][v] -= c_f(p)          forward edge now has less capacity
            G_f[v][u] += c_f(p)          opposite edge has more capacity!
        maxflow += c_f(p)
    return maxflow
```

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

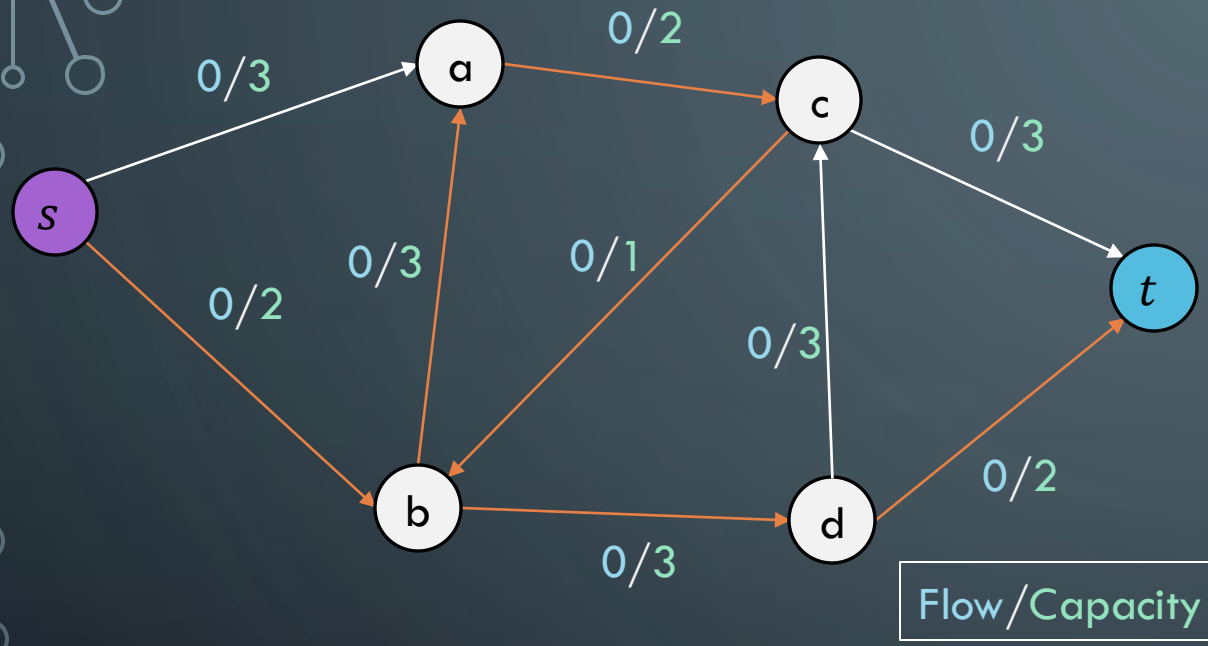
	s	a	b	c	d	t
s	—	3	2	—	—	—
a	0	—	0	2	—	—
b	0	3	—	0	3	—
c	—	0	1	—	0	3
d	—	—	0	3	—	2
t	—	—	—	0	0	—

$c_f(p) = ??$

$maxflow = 0$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

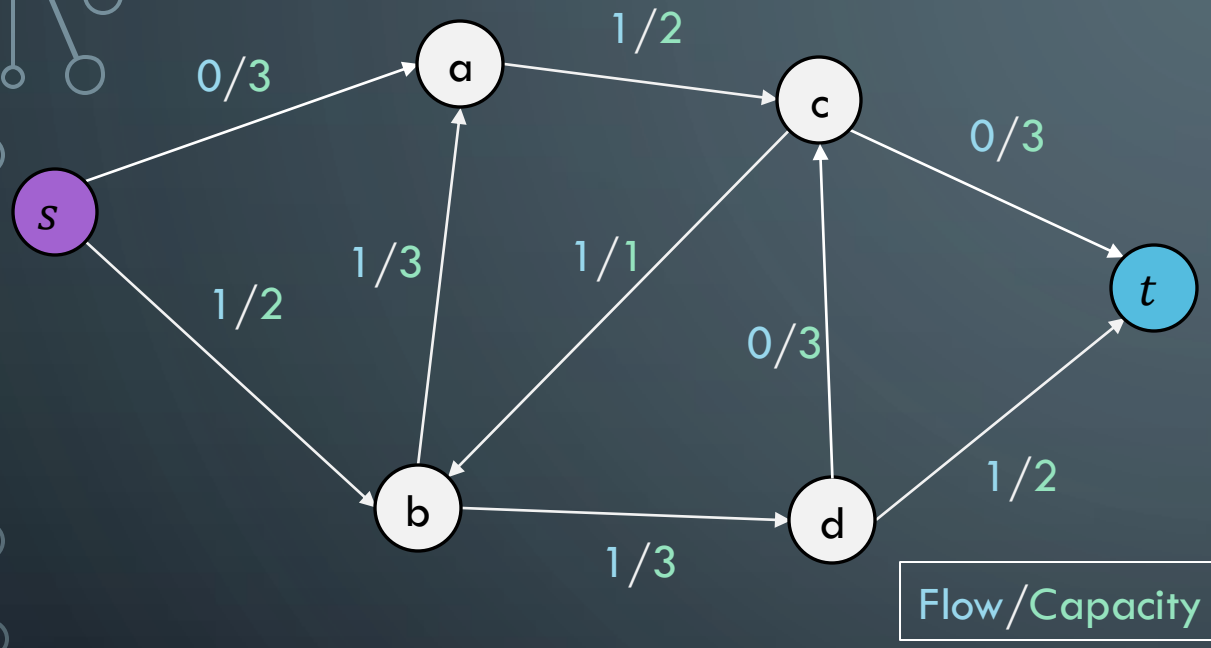
	s	a	b	c	d	t
s	—	3	2	—	—	—
a	0	—	0	2	—	—
b	0	3	—	0	3	—
c	—	0	1	—	0	3
d	—	—	0	3	—	2
t	—	—	—	0	0	—

$$c_f(p) = 1$$

$$maxflow = 0$$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

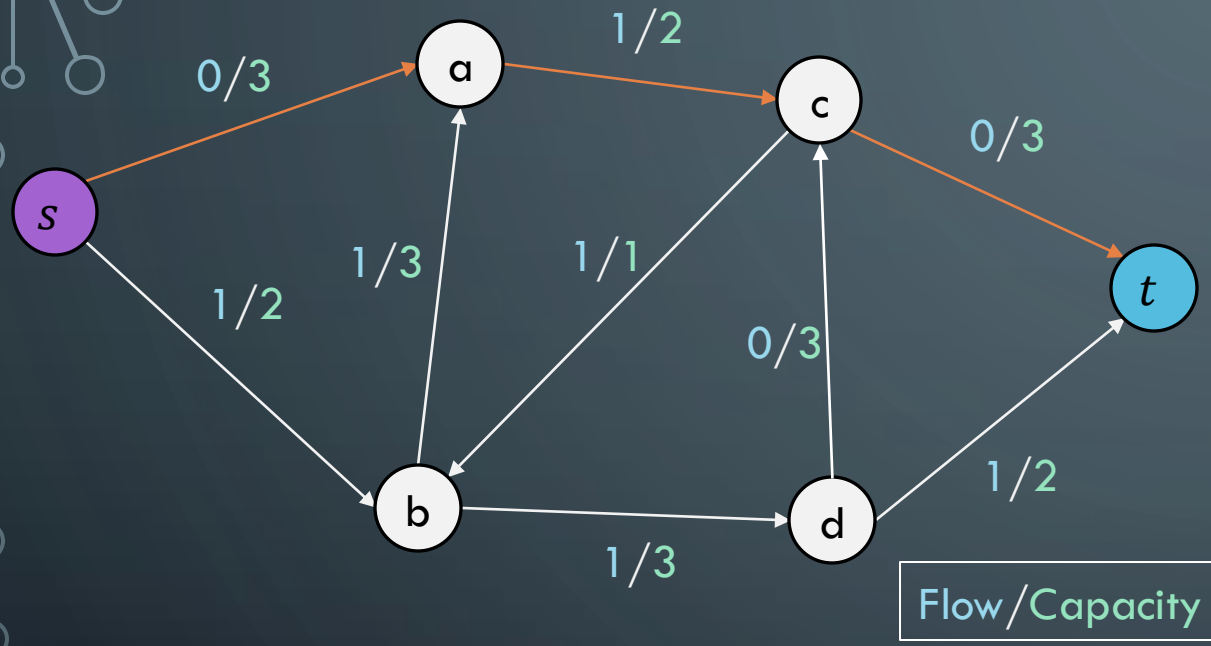
	s	a	b	c	d	t
s	–	3	1	–	–	–
a	0	–	1	1	–	–
b	1	2	–	1	2	–
c	–	1	0	–	0	3
d	–	–	1	3	–	1
t	–	–	–	0	1	–

$c_f(p) = ??$

$maxflow = 1$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

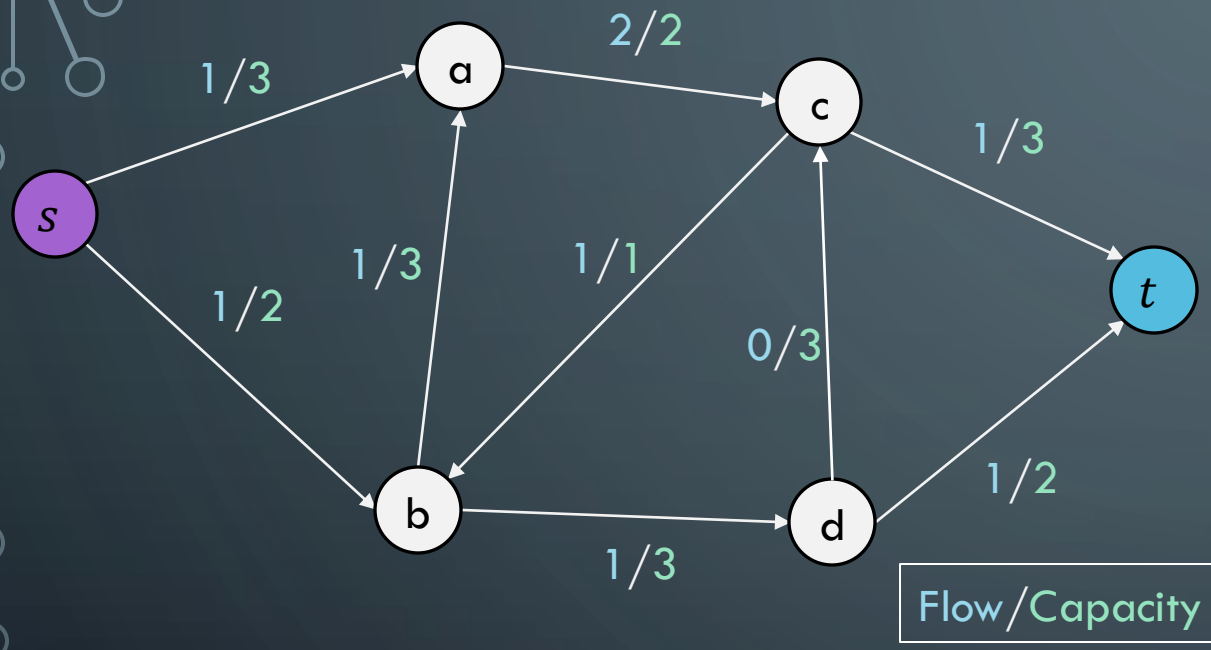
	s	a	b	c	d	t
s	–	3	1	–	–	–
a	0	–	1	1	–	–
b	1	2	–	1	2	–
c	–	1	0	–	0	3
d	–	–	1	3	–	1
t	–	–	–	0	1	–

$$c_f(p) = 1$$

$$maxflow = 1$$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

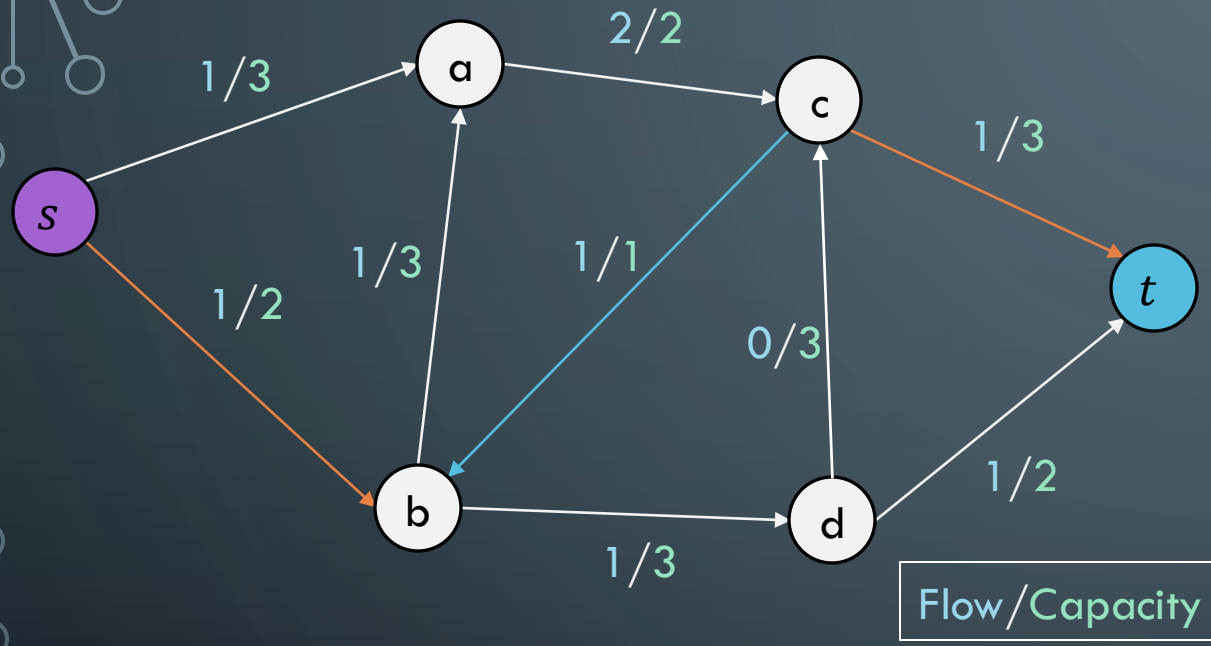
	s	a	b	c	d	t
s	–	2	1	–	–	–
a	1	–	1	0	–	–
b	1	2	–	1	2	–
c	–	2	0	–	0	2
d	–	–	1	3	–	1
t	–	–	–	1	1	–

$c_f(p) = ??$

$maxflow = 2$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

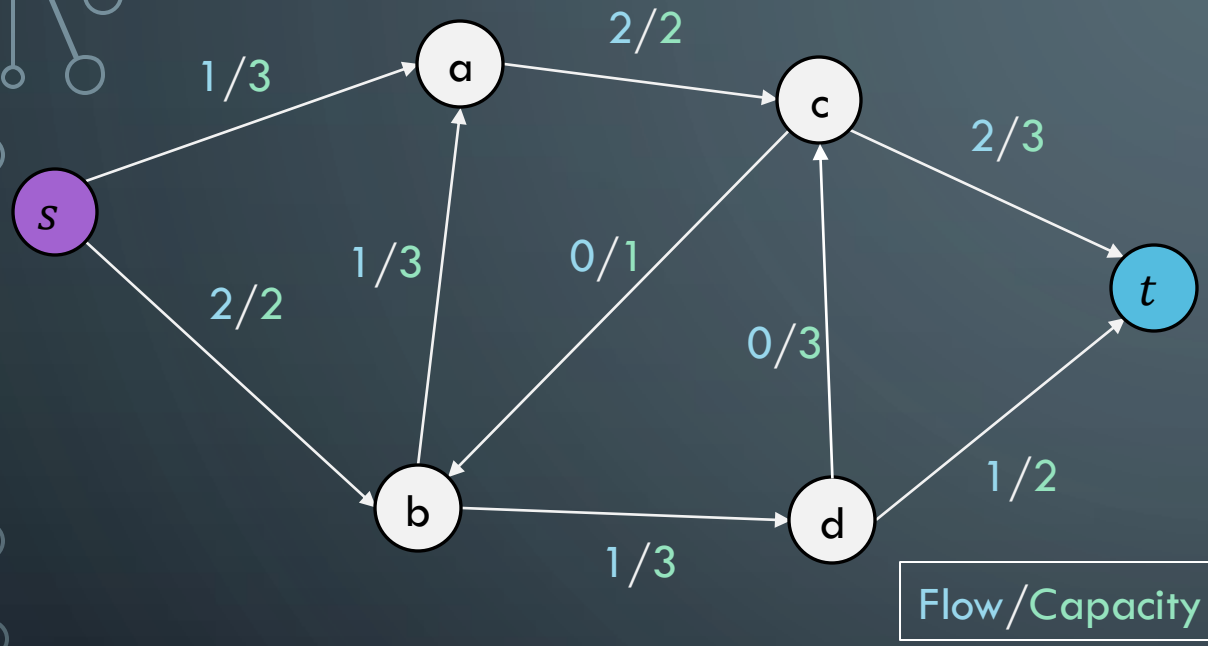
	s	a	b	c	d	t
s	—	2	1	—	—	—
a	1	—	1	0	—	—
b	1	2	—	1	2	—
c	—	2	0	—	0	2
d	—	—	1	3	—	1
t	—	—	—	1	1	—

$$c_f(p) = 1$$

$$maxflow = 2$$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

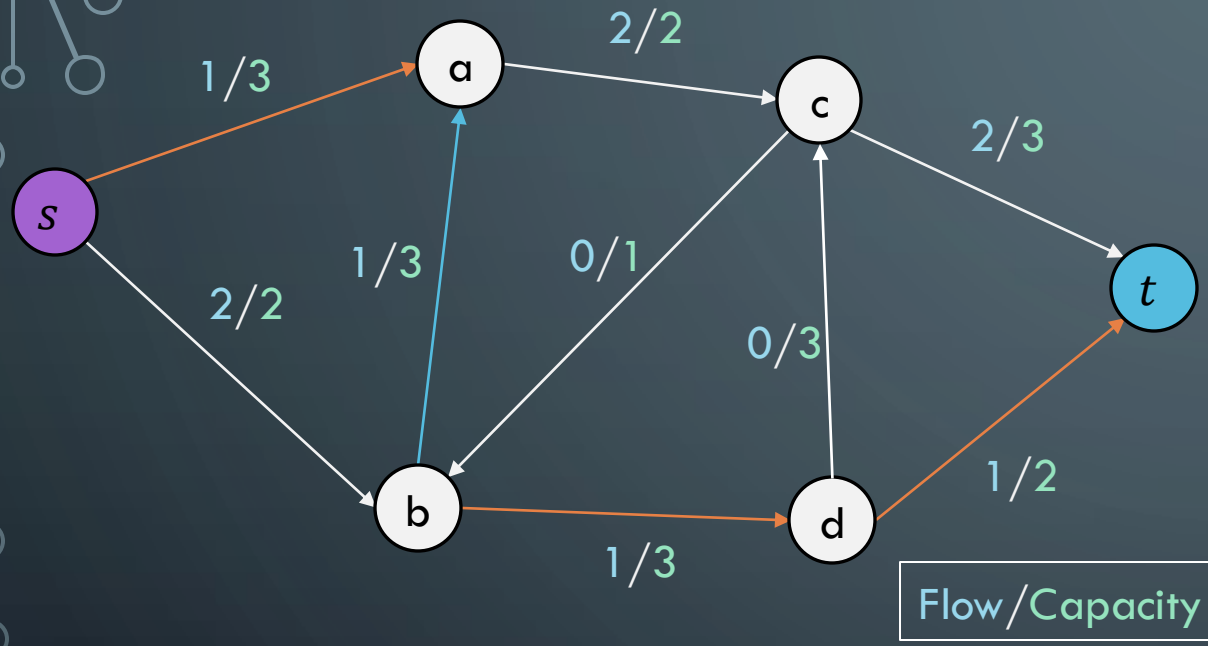
	s	a	b	c	d	t
s	—	2	0	—	—	—
a	1	—	1	0	—	—
b	2	2	—	0	2	—
c	—	2	1	—	0	1
d	—	—	1	3	—	1
t	—	—	—	2	1	—

$c_f(p) = ?$

$maxflow = 3$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

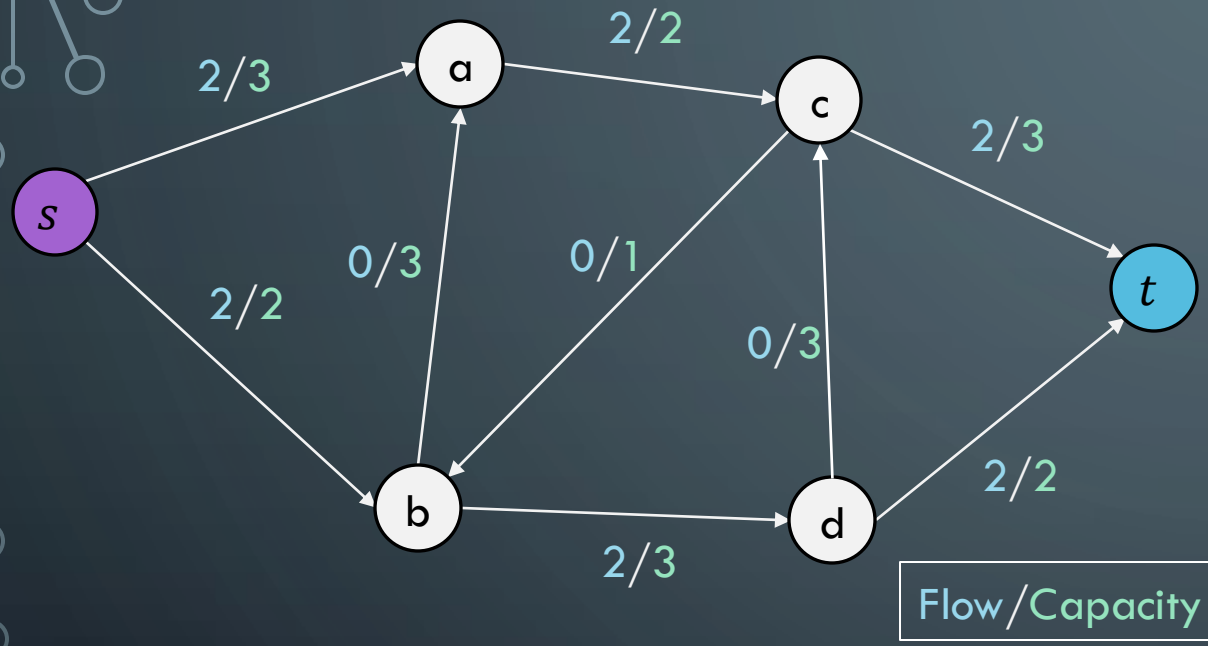
	s	a	b	c	d	t
s	—	2	0	—	—	—
a	1	—	1	0	—	—
b	2	2	—	0	2	—
c	—	2	1	—	0	1
d	—	—	1	3	—	1
t	—	—	—	2	1	—

$$c_f(p) = 1$$

$$maxflow = 3$$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

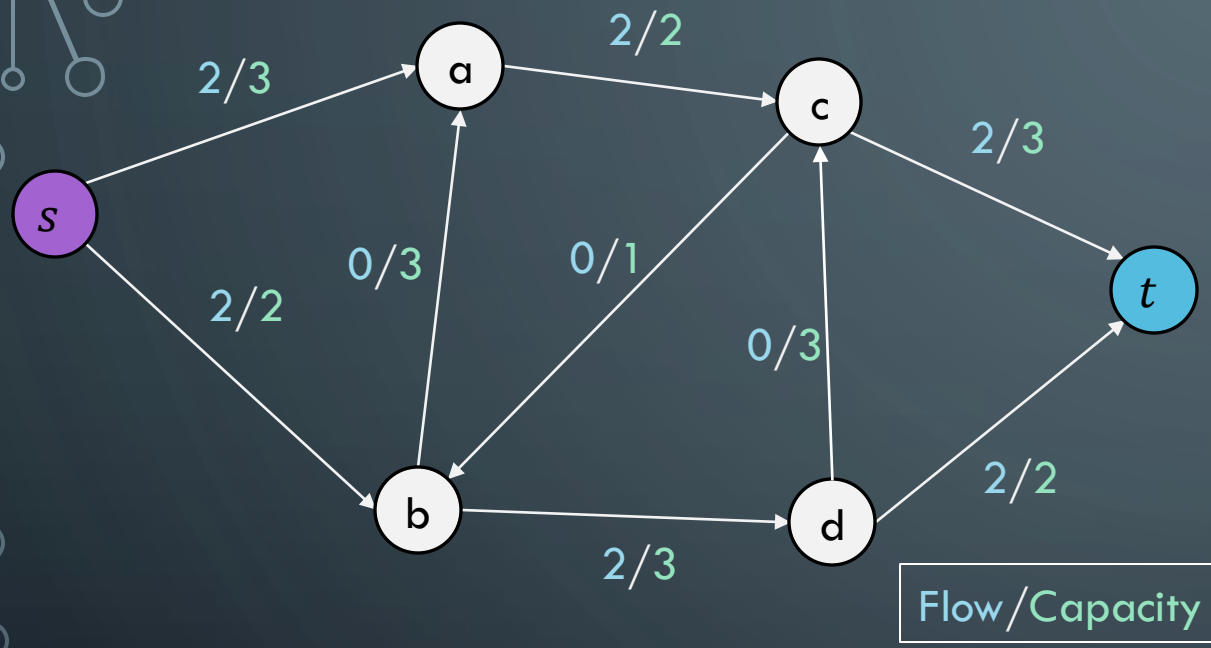
	s	a	b	c	d	t
s	—	1	0	—	—	—
a	2	—	0	0	—	—
b	2	3	—	0	1	—
c	—	2	1	—	0	1
d	—	—	2	3	—	0
t	—	—	—	2	2	—

$$c_f(p) = 1$$

$$maxflow = 4$$

FORD-FULKERSON EXAMPLE

Original Graph



Residual Graph

	s	a	b	c	d	t
s	—	1	0	—	—	—
a	2	—	0	0	—	—
b	2	3	—	0	1	—
c	—	2	1	—	0	1
d	—	—	2	3	—	0
t	—	—	—	2	2	—

No more residual paths, so max flow is 4!!

$$c_f(p) = 1$$

$$maxflow = 4$$

RUNTIME

```
Ford-Fulkerson(G, s, t):
    Construct residual graph G_f with f(u,v) = 0 for all edges
    maxflow = 0
    while augmenting paths exist:
        Run DFS to find path p in G_f | c_f(u,v) > 0 for all edges (u,v) ∈ p
        Find c_f(p) = min{c_f(u,v) | (u,v) ∈ p}
        for each edge (u,v) ∈ p:
            G_f[u][v] -= c_f(p)          forward edge now has less capacity
            G_f[v][u] += c_f(p)          opposite edge has more capacity!
        maxflow += c_f(p)
    return maxflow
```

DFS is $\Theta(V + E)$, worst case is we update flow by only 1 each time.
Total: $\Theta\big(|f|(V + E)\big) = \Theta(E \cdot |f|)$

WHAT TYPE OF SEARCH?

- "While there is an augmenting path p in G_f"
  - Using a depth-first search is the Ford-Fulkerson algorithm
    - Each augmenting path can be found in O(E) time
    - And there can be $|f|$ paths
    - So the running time is O($E \cdot |f|$)
    - Will not terminate with irrational edge values
  - Using a breadth-first search is the Edmonds-Karp algorithm
    - Runs in $O(V \cdot E^2)$
      - Total number of augmentations is O(V·E)
      - And finding each augmentation takes O(E)
    - Guaranteed termination with irrational edge values
    - Run-time is independent of the maximum flow of the graph

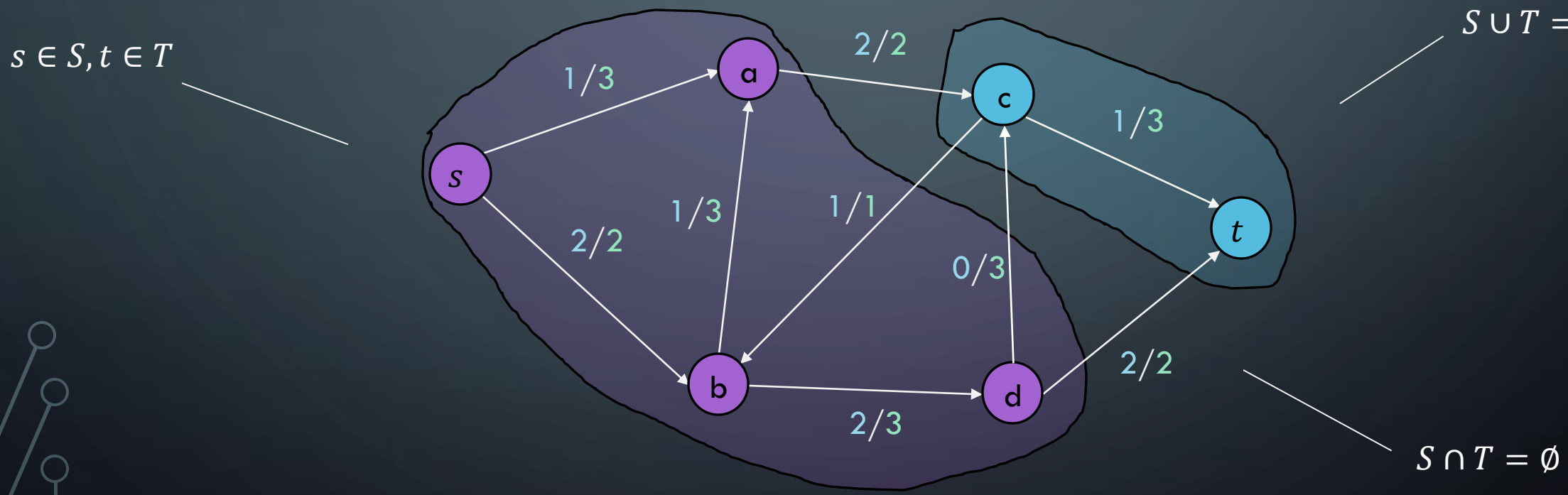
MIN-CUTS AND PROVING CORRECTNESS OF FORD-FULKERSON

SHOWING CORRECTNESS OF FORD-FULKERSON

- To prove Ford-Fulkerson correct
  - We'll use the idea of a **cut** in a network flow graph
  - We'll prove properties related to cuts and flows

CUTS IN NETWORK FLOW GRAPHS

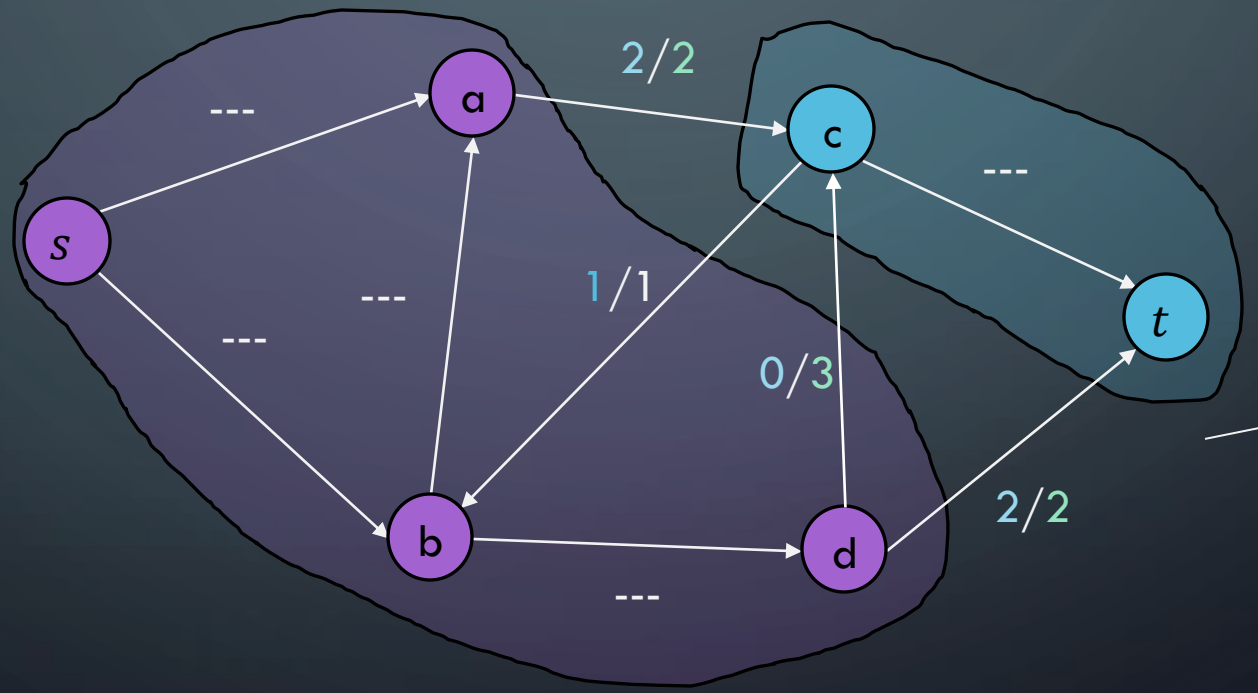
A cut C = (S, T) is an assignment of the vertices of a flow-network each to one of two groups S and T.



PROPERTIES OF A CUT

The ***capacity*** of a cut Cap(C) where C = (S, T) is the maximum amount of flow that COULD cross the cut.

The ***net flow*** of a cut NF(C) C = (S, T) is the net amount of flow that is crossing the cut.



Capacity of C is:
2+3+2 = 7

Net flow is:
2+2-1=4

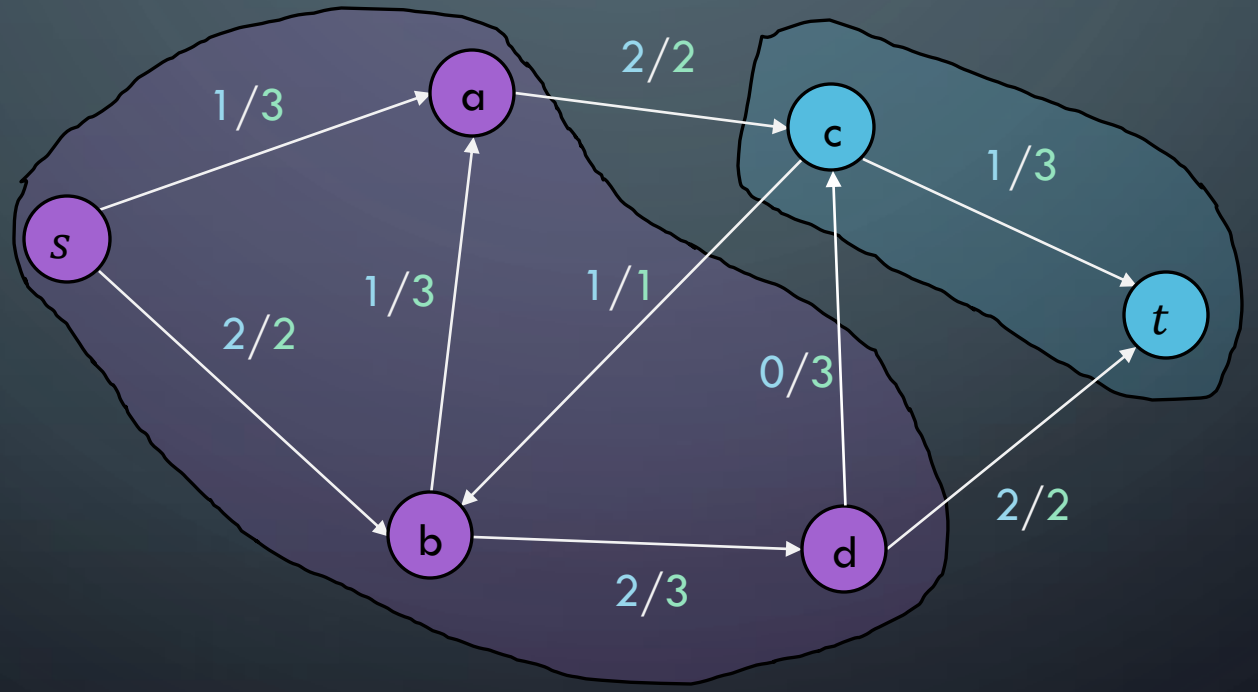
WE'LL SHOW THESE 3 THINGS

- Weak duality property
- Flow-value lemma
- Max-flow Min-Cut theorem

- …and how we can then argue Ford-Fulkerson is correct

FIRST DEFINITION: WEAK DUALITY

Let f be any flow and C = (S, T) be any cut.
Then it must be the case that $f \leq Cap(C)$



Intuitively, flow needs to get across the cut at some point, so the capacity of the cut must be high enough to accommodate this!

DEFINITION: FLOW-VALUE LEMMA

Let f be the flow and C = (S, T) be any cut.
Then it must be the case that $\mathrm{NF}(C) = f$

The net flow across a cut is always exactly the flow of the network

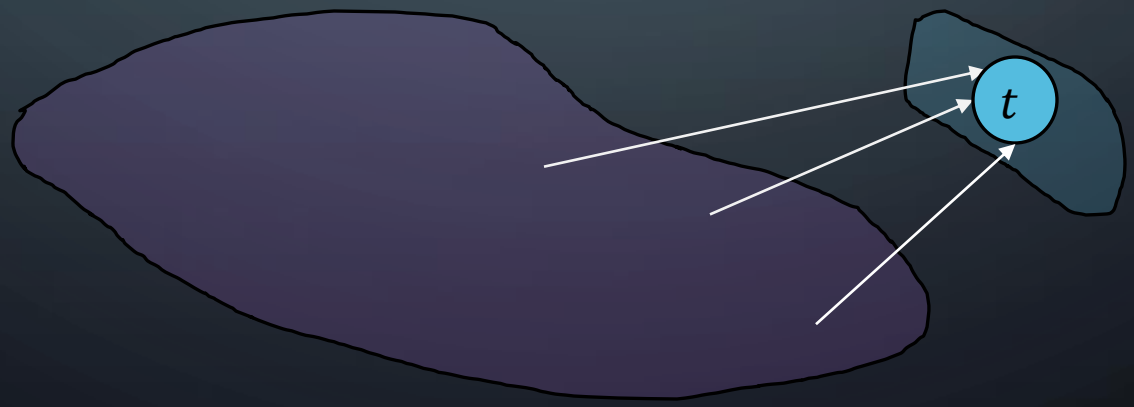
PROOF: FLOW-VALUE LEMMA

Let f be the flow and C = (S, T) be any cut.
Then it must be the case that $\mathrm{NF}(C) = f$

The net flow across a cut is always exactly the flow of the network

Proof by induction on the size of T

Base Case: $T = \{t\}$



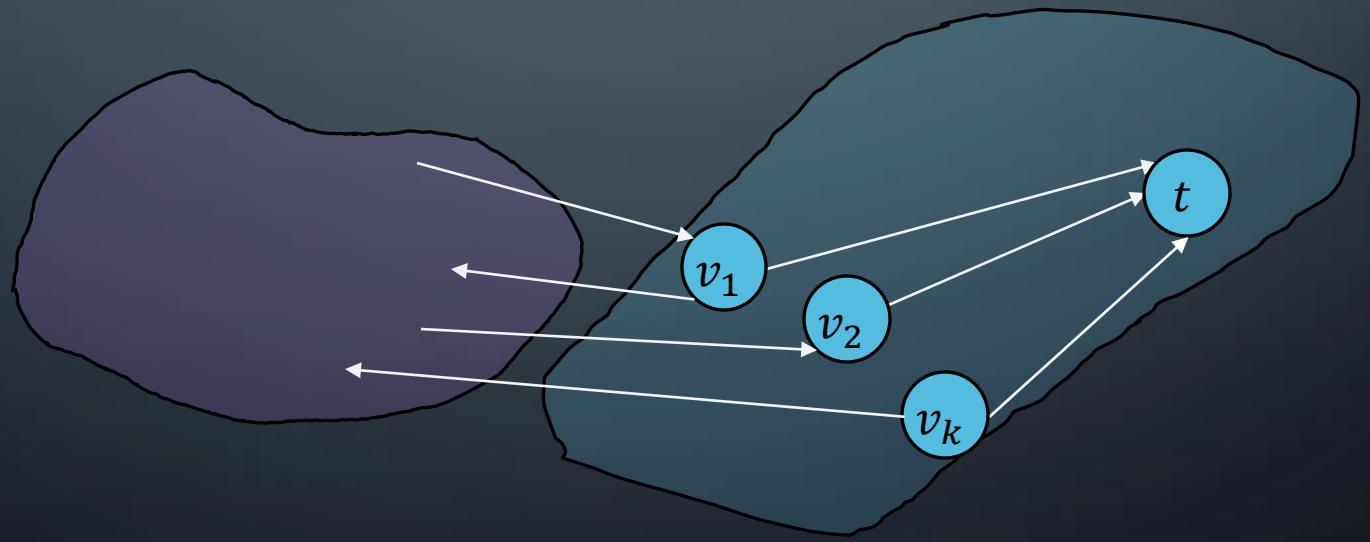
Trivially true, the net-flow of this cut is the flow of the network by definition

PROOF: FLOW-VALUE LEMMA

Let f be the flow and C = (S, T) be any cut.

Then it must be the case that $\mathrm{NF}(C) = f$

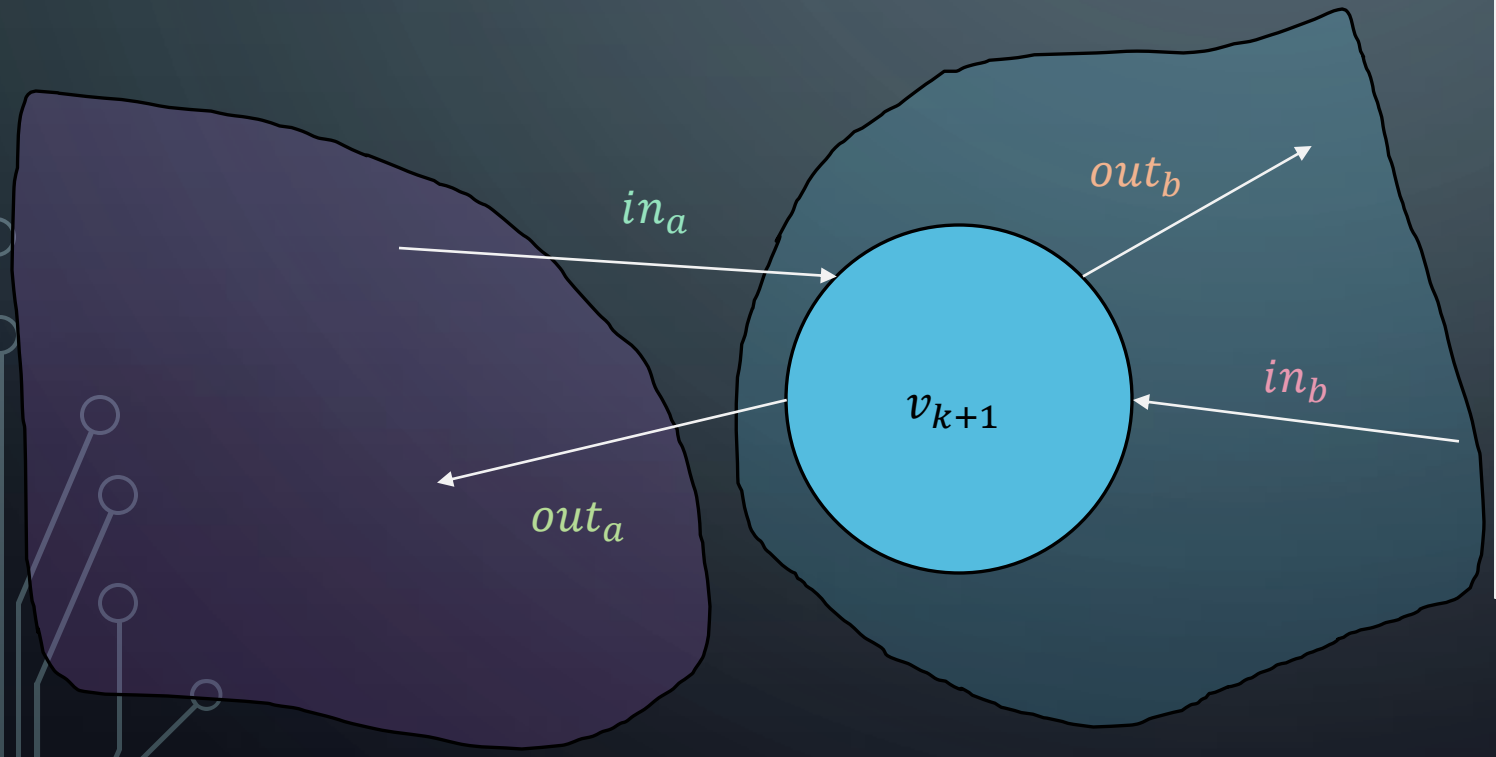
Inductive Hypothese: Assume theorem true for $T = \{t, v_1, \ldots, v_k\}$ for some k additional nodes



PROOF: FLOW-VALUE LEMMA

Let f be the flow and C = (S, T) be any cut.
Then it must be the case that $\mathrm{NF}(C) = f$

Inductive Step: Move one more node across the cut. How does flow across cut change?



$$NF(C) = f$$

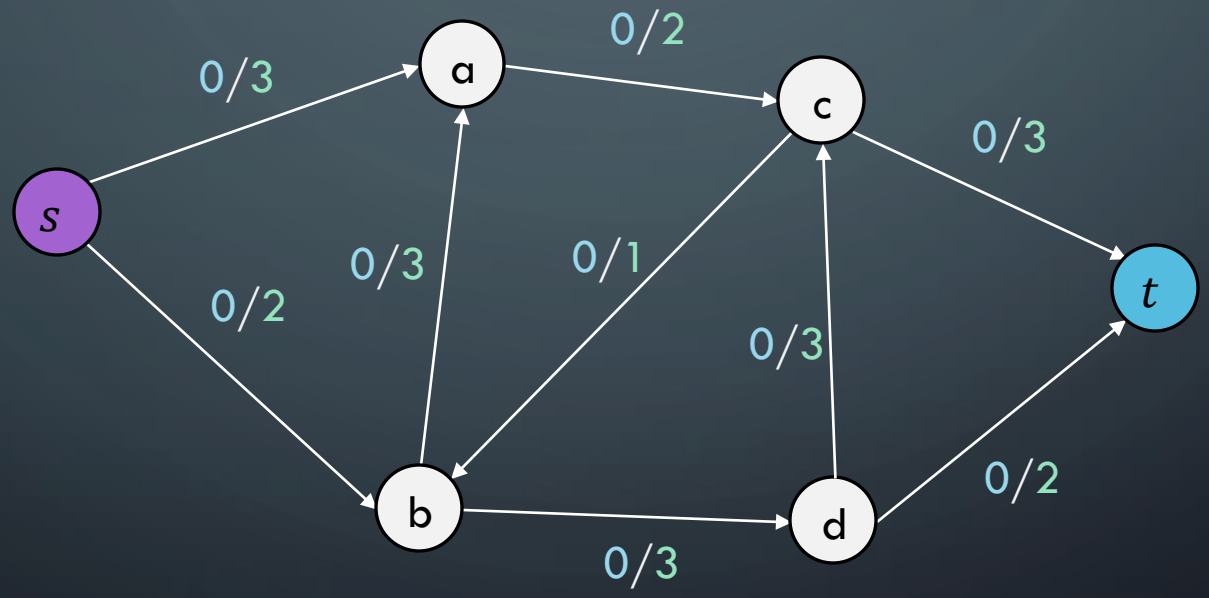
$$NF(C') = NF(C) - out_b - out_a + in_a + in_b$$
$$NF(C') = NF(C) + (in_a + in_b - out_b - out_a)$$
$$NF(C') = NF(C) + (totalin - totalout)$$
$$NF(C') = NF(C) + 0)$$
$$NF(C') = NF(C) = f$$

MAX-FLOW MIN-CUT THEOREM

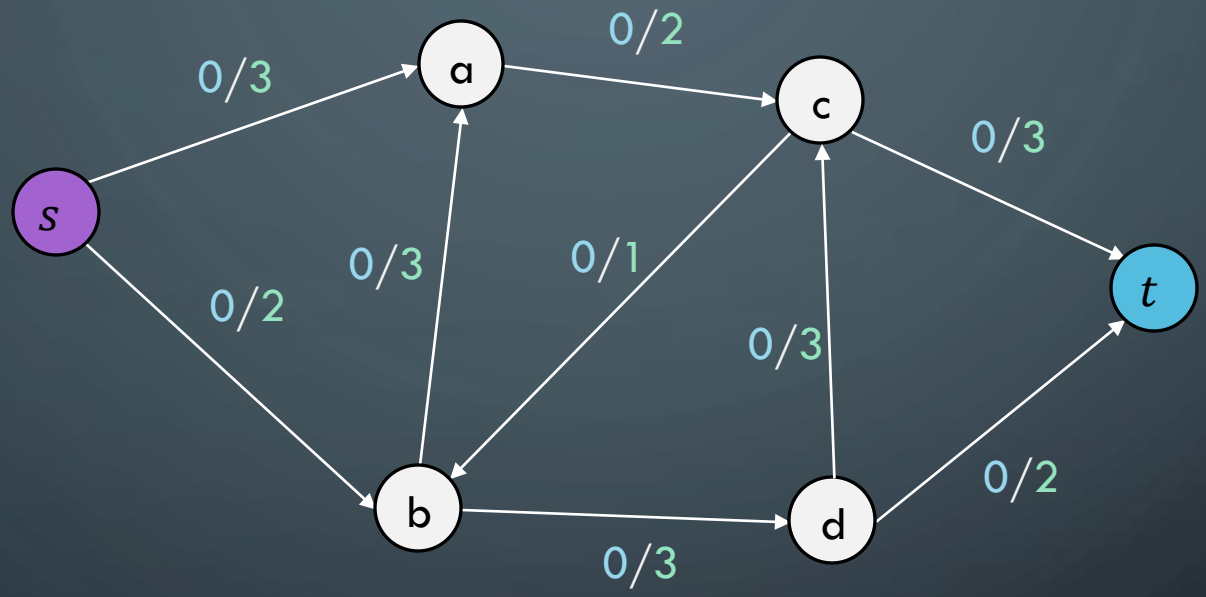
The Max-flow Min-cut theorem: the maximum value of an s-t flow is equal to the minimum capacity of an s-t cut

MAX-FLOW MIN-CUT THEOREM

The Max-flow Min-cut theorem: the maximum value of an s-t flow is equal to the minimum capacity of an s-t cut



Let's draw some cuts and figure out which is the minimum capacity



FORD-FULKERSON PROOF

We'll show the following three statements are equivalent, and together this shows Ford-Fulkerson is correct.

A) There is no augmenting path in G_f with respect to f

B) There exists a ***cut*** in G whose capacity equals f

C) The flow f is a maximum flow in G

We will do the following three mini-proofs:

- A → B
- B → C
- C → A

Why do these three statement imply that ford-fulkerson works correctly and always finds the maximum flow?

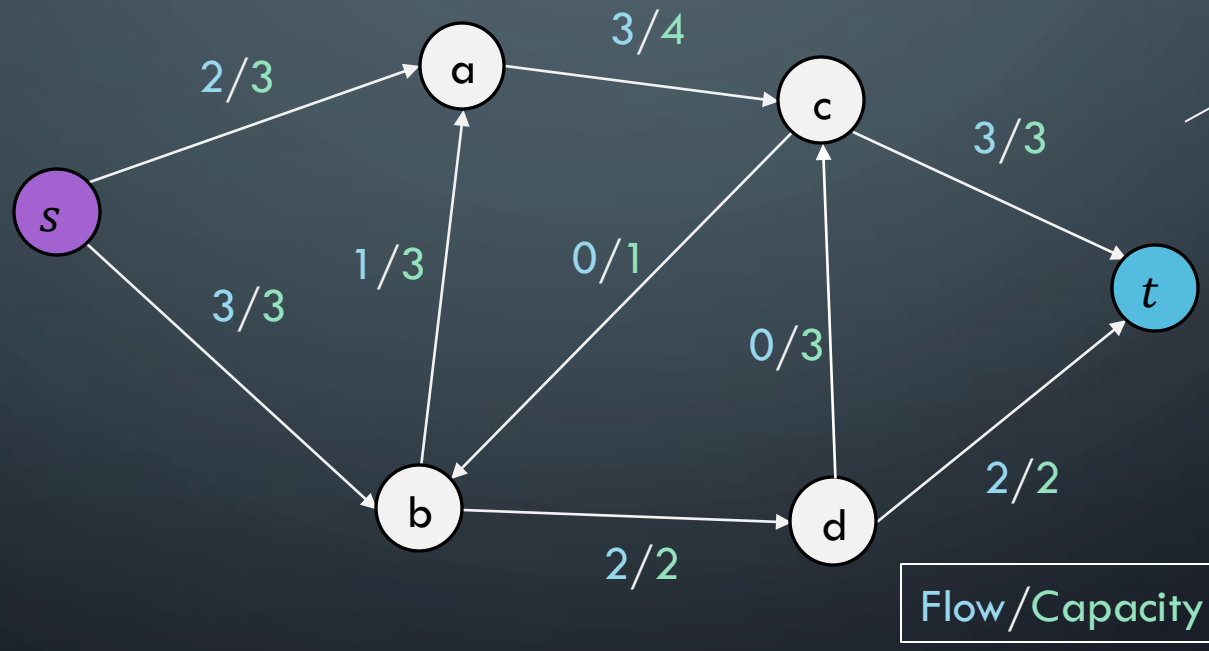
A → B: IF NO AUGMENTING PATH, CUT WITH CAPACITY F

A) There is no augmenting path in G_f with respect to f

B) There exists a ***cut*** in G whose capacity equals f

A → B: IF NO AUGMENTING PATH, CUT WITH CAPACITY F

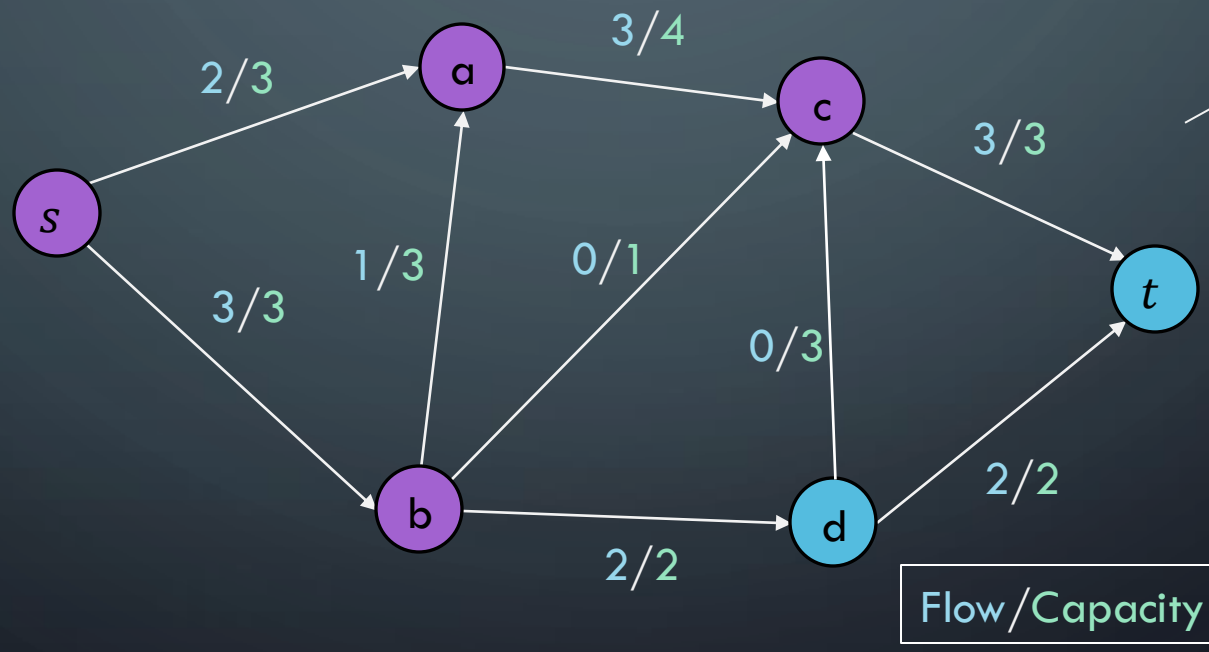
Step 1) Assume that graph G has no augmenting path



Proof is general, but here is one example. Graph has no augmenting paths

A → B: IF NO AUGMENTING PATH, CUT WITH CAPACITY F

Step 2) Find nodes N in G_f reachable via DFS, let the cut $C = (N, V - N)$

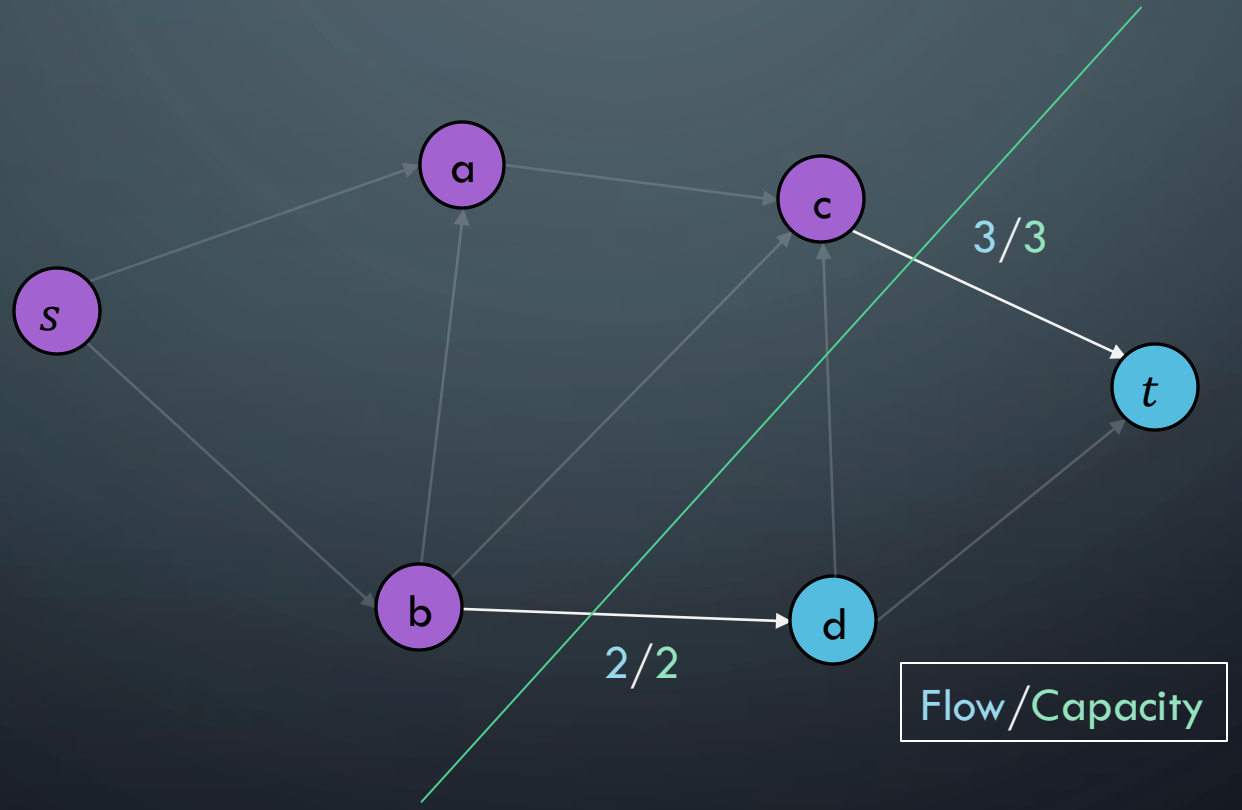


Purple nodes are reachable. Note one backflow edge is used here

A → B: IF NO AUGMENTING PATH, CUT WITH CAPACITY F

Step 2) Find nodes N in G_f reachable via DFS, let the cut $C = (N, V - N)$

Step 3) Cut C is guaranteed to be at capacity. $cap(C) = f$



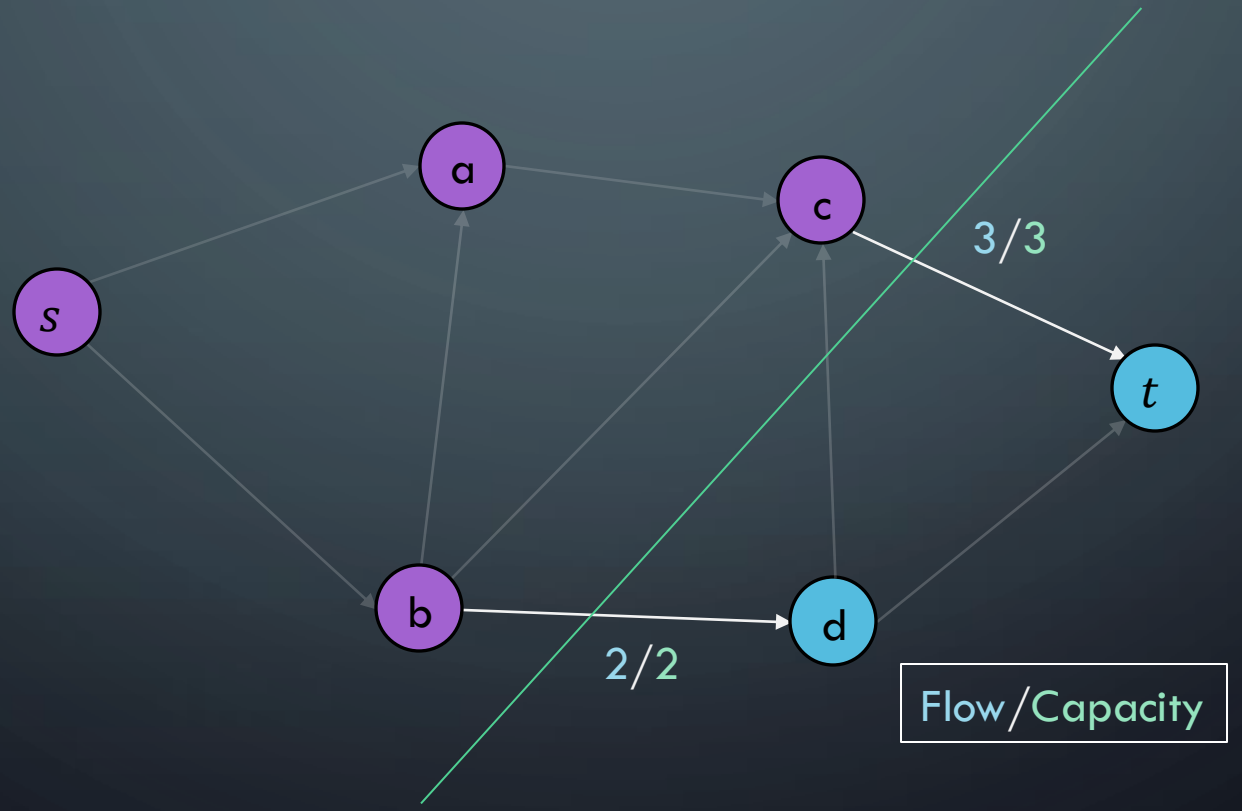
B → C: IF CUT WITH CAPACITY F, THEN MAX FLOW

B) There exists a ***cut*** in G whose capacity equals f

C) The flow f is a maximum flow in G

B → C: IF CUT WITH CAPACITY F, THEN MAX FLOW

Step 1) Assume that graph G_f *has a cut C with* $cap(C) = f$

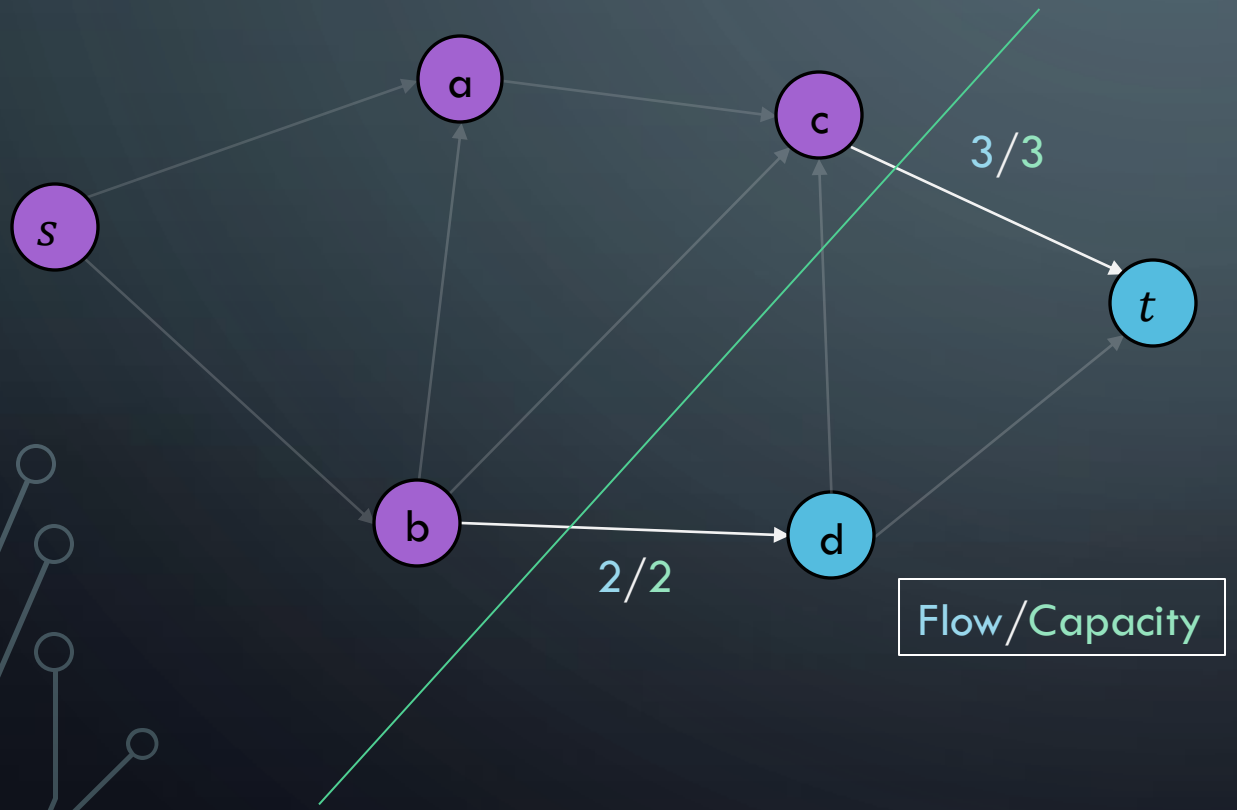


B → C: IF CUT WITH CAPACITY F, THEN MAX FLOW

In this example, we somehow push more than 5 flow through this graph.

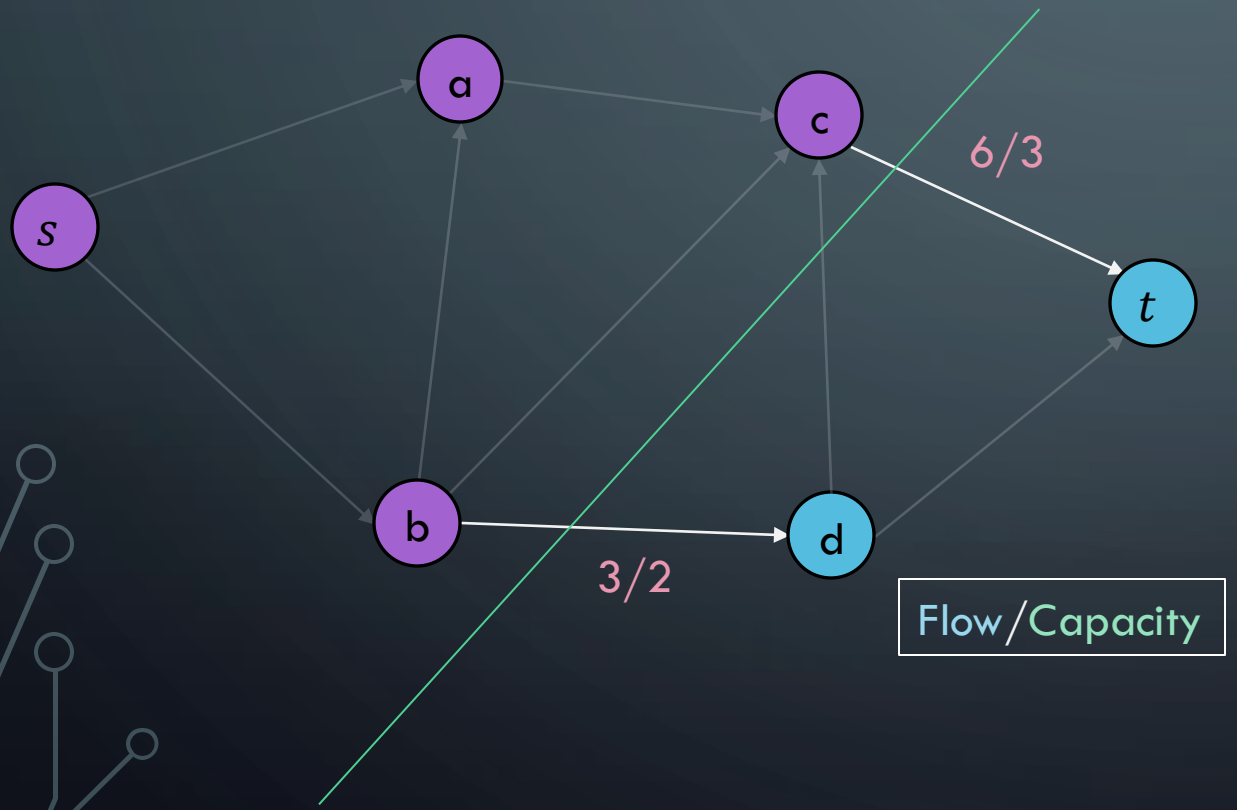
Assume f.s.o.c that f not maximum

Thus, some other flow $f' > f$ is better



B → C: IF CUT WITH CAPACITY F, THEN MAX FLOW

Thus, you are somehow pushing more flow across this cut than is possible (over capacity)



Assume f.s.o.c that f *not maximum*

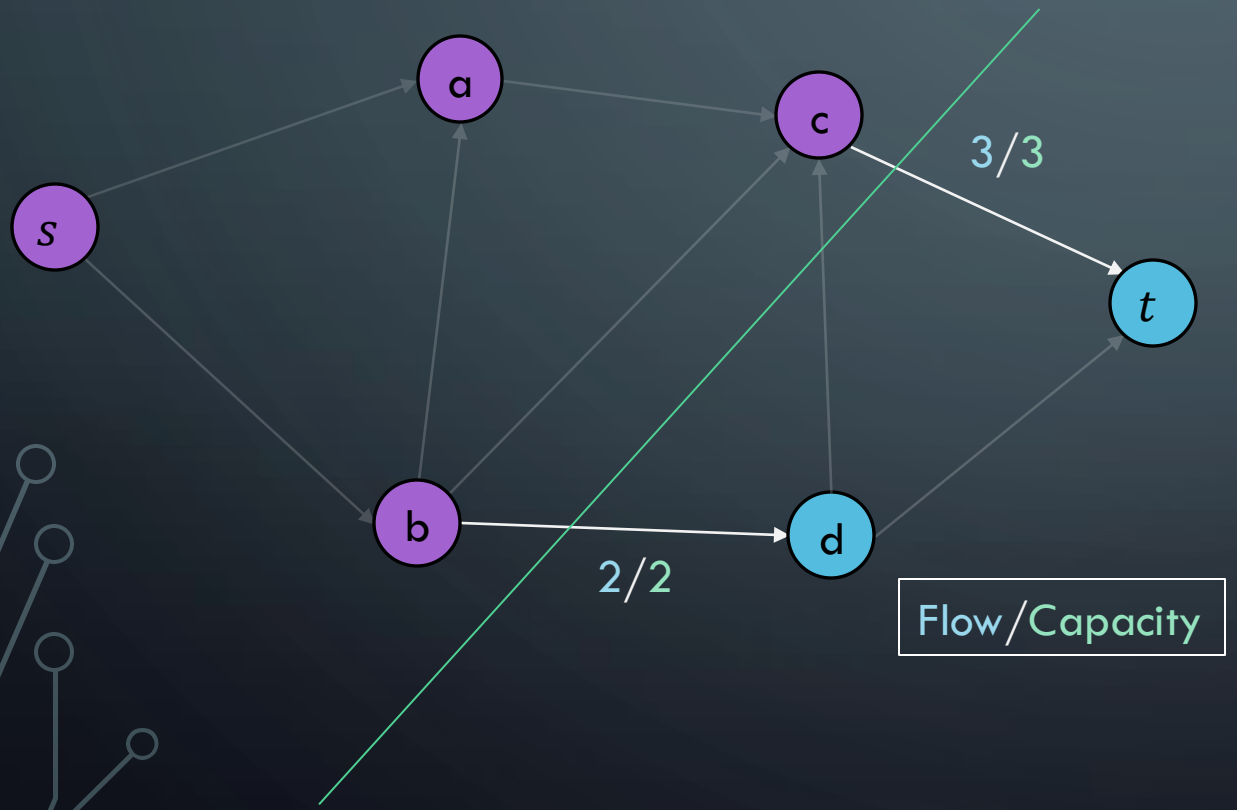
Thus, some other flow $f' > f$ *is better*

Flow-Value Lemma: $NF(C) = f'$

$$f' > f = cap(C)$$

$$f' > cap(C)$$

B → C: IF CUT WITH CAPACITY F, THEN MAX FLOW



Assume f.s.o.c that f *not maximum*

Thus, some other flow $f' > f$ *is better*

Flow-Value Lemma: $NF(C) = f'$

$$f' > f = cap(C)$$

$$f' > cap(C)$$

***Contradiction**: This violates weak-duality, which states that flow never exceeds capacity of a cut.*

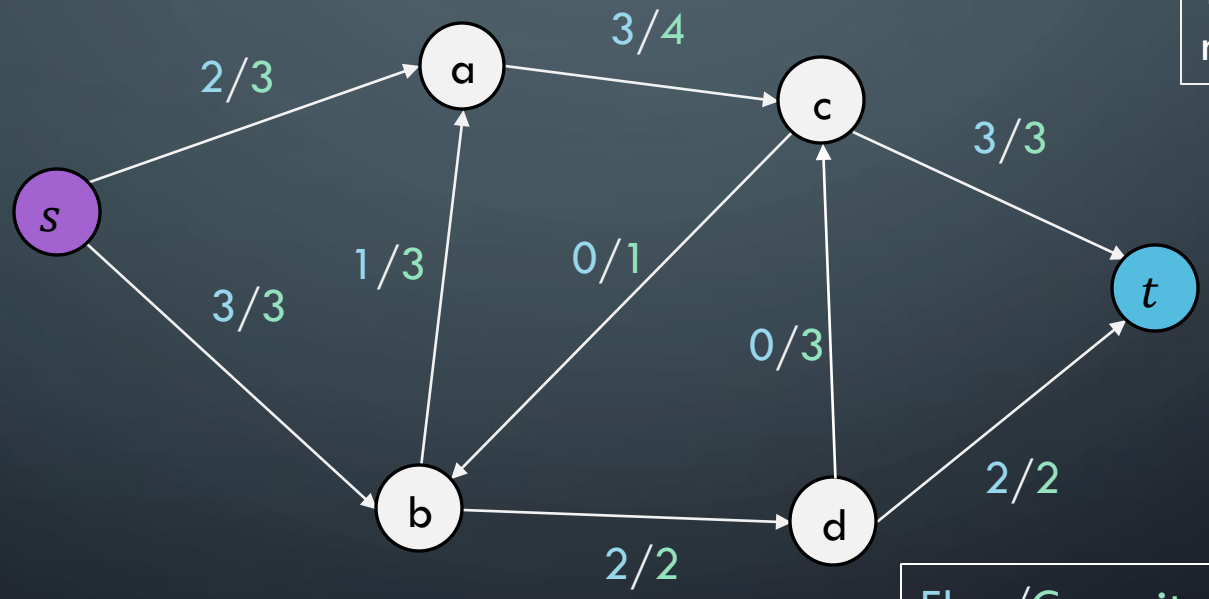
C → A: IF F IS THE MAX FLOW, THEN NO AUGMENTING PATH

A) There is no augmenting path in G_f with respect to f

C) The flow f is a maximum flow in G

C → A: IF F IS THE MAX FLOW, THEN NO AUGMENTING PATH

Step 1) Assume that f is the maximum flow



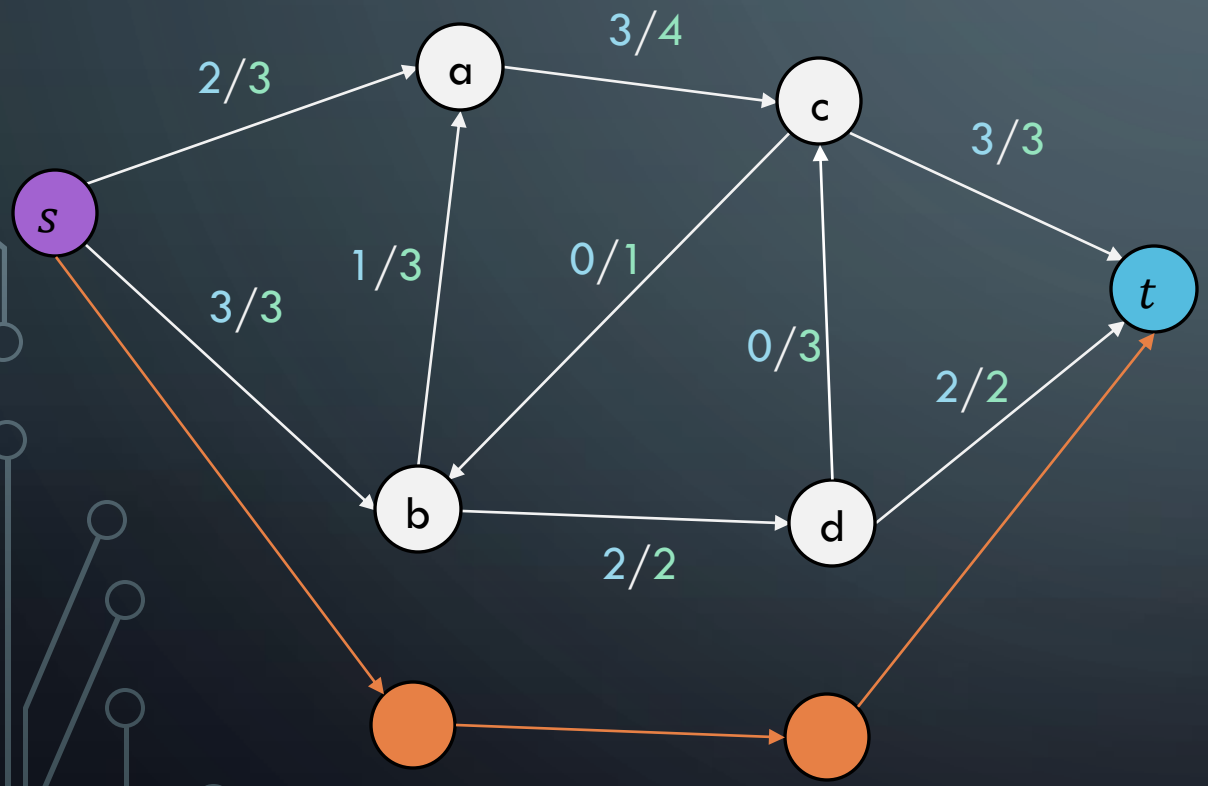
This graph is at maximum flow

Flow/Capacity

C → A: IF F IS THE MAX FLOW, THEN NO AUGMENTING PATH

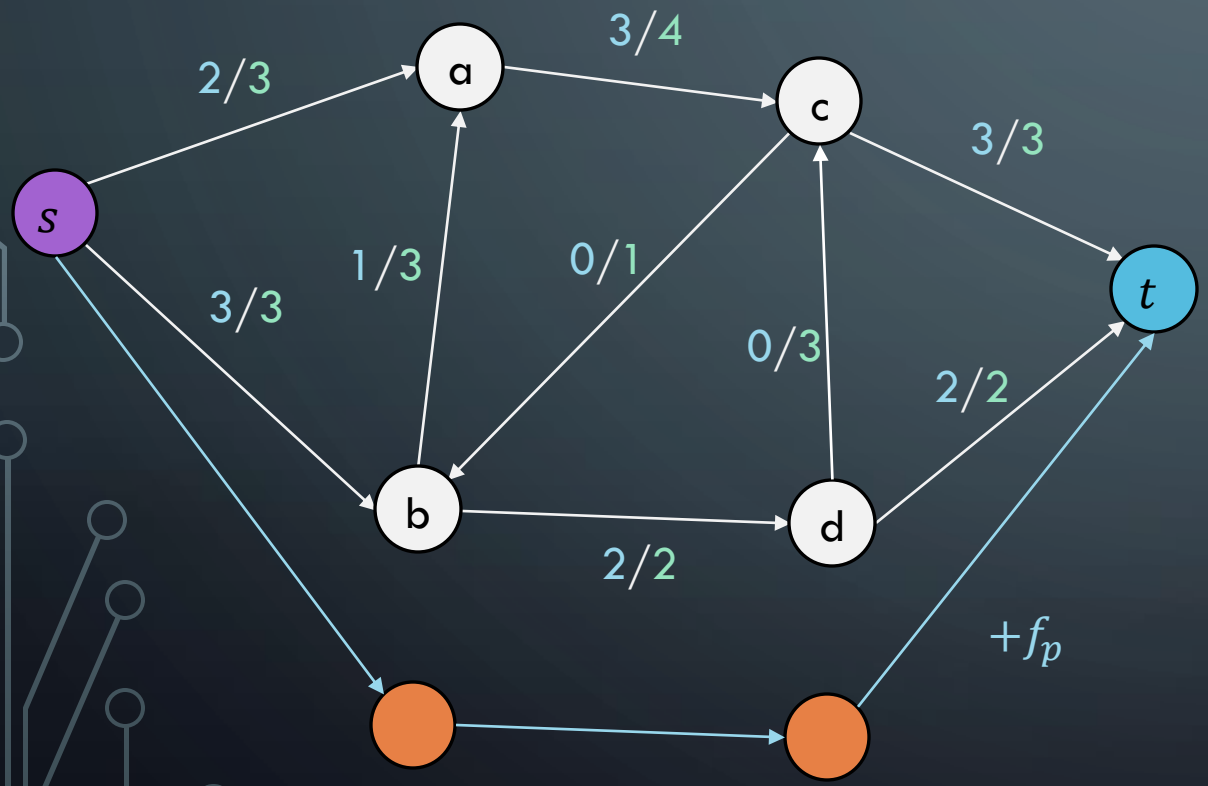
There isn't, but let's assume there is…

Assume f.s.o.c that there IS an augmenting path in G_f



C → A: IF F IS THE MAX FLOW, THEN NO AUGMENTING PATH

There isn't, but let's assume there is…



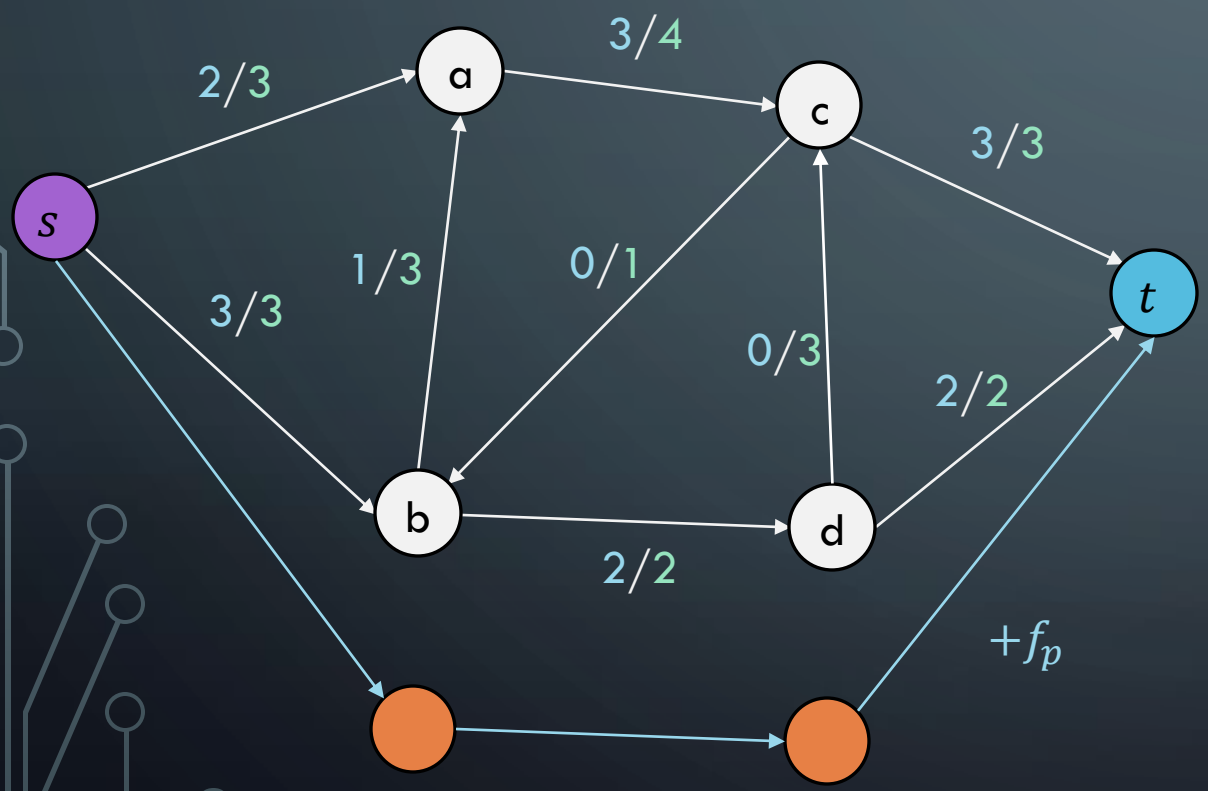
Assume f.s.o.c that there IS an augmenting path in G_f

Then, we can push more flow $f_p > 0$ through the network

Thus, new flow becomes $f + f_p > f$

C → A: IF F IS THE MAX FLOW, THEN NO AUGMENTING PATH

There isn't, but let's assume there is…



Assume f.s.o.c that there IS an augmenting path in G_f

Then, we can push more flow $f_p > 0$ through the network

Thus, new flow becomes $f + f_p > f$

Contradiction: f was stated to be maximum flow

FORD-FULKERSON PROOF

Thus, it is proven!

A) There is no augmenting path in G_f with respect to f

B) There exists a ***cut*** in G whose capacity equals f

C) The flow f is a maximum flow in G

OTHER MAX-FLOW ALGORITHMS

- **Ford-Fulkerson**
  - $\Theta(E|f|)$
- **Edmonds-Karp**
  - $\Theta(E^2V)$
- Push-Relabel (Tarjan)
  - $\Theta(EV^2)$
- Faster Push-Relabel (also Tarjan)
  - $\Theta(V^3)$